Learn LLM Inference from Scratch by building a small vLLM by hand using C++ and CUDA.

Copyright (c) 2026 Samir Sarwar. All rights reserved. Personal reading and study are permitted; redistribution requires prior written permission.

Table of Contents

Preface: **2**
What am i doing: **2**
Wtf is inference?: **2**
Chef Anlogy: 3
Why C++ and CUDA: 4
SafeTensors: 4
What we implement: 5
An aside on memory: 10
Loading the Model: 10
Tokenization **14**
Normilization using RMSNorm: **16**
Understanding the code: 18
RoPE (Rotary Positional Embedding/Encoding): **21**
Attention: **29**
Single Headed Attention: 29
Multi-headed Attention: 33
Grouped Query Attention: 35
Feed Forward Network: **36**
Decode: **48**
Prefill vs.Decode: 48
Why cache?: 50
Batching: **61**
Latency and Throughput: 61
Buffer reuse: 63
Decode step walkthrough: 66
Static Batching: 70
Continuous Batching: 72
Req identity 73
Finished req slots: 74
Memory: 77
Paged KV Cache & Attention **79**
Conclusion: **86**

Preface:

If you've made it here, welcome! Below you'll find 80+ pages of handtyped notes on the basics of LLM inference from the ground up for complete beginners. It is quite informal, more similar to a friend explaining a concept to you rather than a textbook. All text below was written by me, **NO AI GENERATION** so I hope that you can enjoy the fruits of my labour. There are accompanying videos that go along with these notes as part of a youtube video series building out the entire inference engine from scratch if you're more interested in the coding side. If you do want more structured textbook style notes, those are also available in the github repo, though those are AI generated based off my videos and notes. Yes there are typos, and short-formed versions of words I apologize :(.

What am i doing:

Building an LLM inference engine with C++ and CUDA similar to a much smaller version of VLLM. VLLM is an open source LLM inference engine which supports many models.

Wtf is inference?:

Well the most simple answer to this is that AI inference is when a model provides an answer based on data.

Where a model is practically a file with weights that are represented as floating point numbers, with operations/functions that are using these weights. Operations are functions that take data as input and output some other data, but more specifically it's the mathematical steps a model performs like matrix math.

An LLM is a Model. One with billions of weights that let it predict the next word or token in text.

Weights are learned during training, and that includes post-training as well, like instruction tuning where we're still updating the weights to change how the model responds. Once we take that resulting

checkpoint and use it for inference we keep the weights fixed, so the numbers changing with each input are the hidden states and other results we calculate, we aren't training the model again every time someone asks it something.

So what we're getting at here is that inference handles how we USE the model, so during training the model learns itself and discovers its weights and now that we have that, inference uses that model on a specific input to produce a prediction

Where an inference engine is the software that performs the process above efficiently. So putting the input through the math, using GPU, managing memory, and getting result

Chef Anlogy:

A good way to think about it thats helping me wrap my head around this is a chef whose learned millions of recipes.

Training: You can say culinary school, this is where they learned the recipes by seeing whats right, wrong, and correcting itself. By the end they know what certain amount, and type of ingredients will create what dish, These would be the weights. **The Model** is thus the chef's learned skill and knowledge

Inference: Is asking the chef to make a meal with some ingredients and the chef producing an answer, so see this is quite abstracted. Give him some shi, and see what he cooks up (literally). Inference is the actual act of cooking.

Inference Engine: The kitchen; providing pretty much everything needed for the chef to execute the meal based on the input, and based on what they've learned from school. The better this is, the more efficient. I.e it affects speed, memory use, cost, and how many requests it can handle. This is easy analogous to kitchen being the inference engine as well. The model determines what the chef has learned, and making the kitchen faster should still let us carry out the same recipe, but the way we set things up can affect the answer as well, like how we format the prompt, how much context we keep, the precision we

calculate with and how we choose the next token. Our engine for example always picks the highest-scoring token, so even with the same model weights those choices can change what we end up generating.

Thus you may ask, why do we need an inference engine beyond optimizations? Well you can't run the file with the weights of the model, this is not executable, its just a bunch of numbers. The architecture of the model also can't run its just a blueprint and not that useful without the model being trained by data. And so we must Serve the model meaning writing a program that implements the operations of the model.

Why C++ and CUDA:

Well, we wanna be fast, and optimize hardware as best as possible, more specifically the GPU. C++ is extremely performant with insane customizabiltiy with optimizations, and CUDA allows us to program GPU hardware and memory. Why GPU? Remember above when I mentioned that the cook needs to actually create the meal during inference, and so it needs to perform all of the model operations which is a shit ton of matrix multiplications that are handled much better on GPUs rather than CPUs as they have more powerful processing cores.

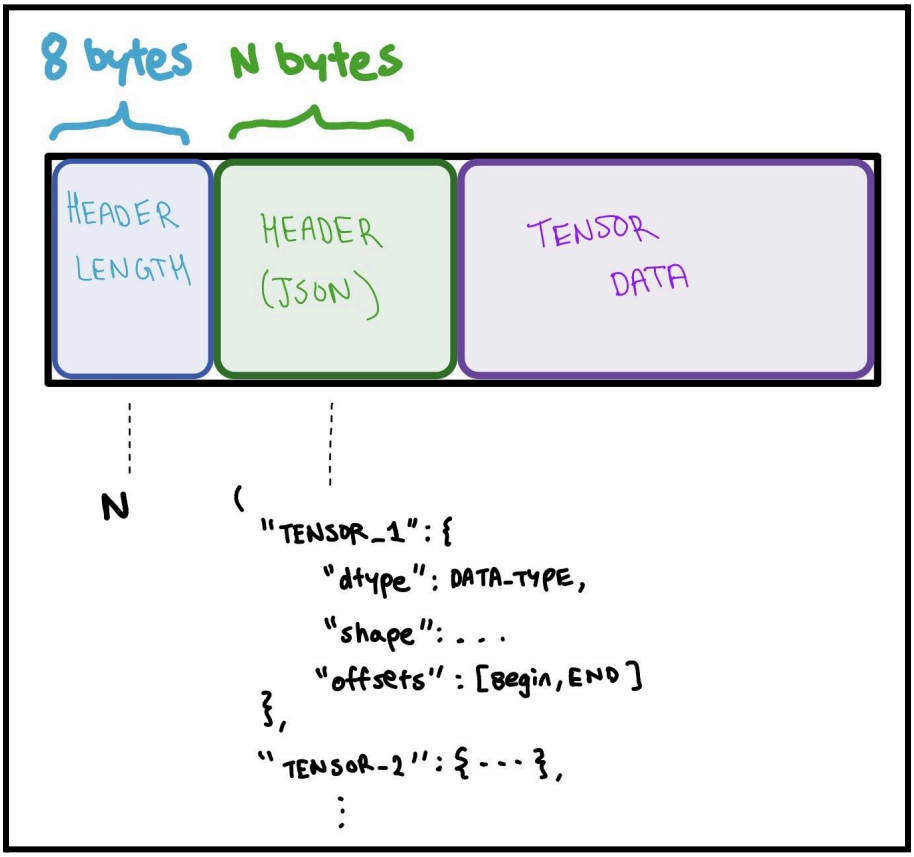
SafeTensors:

There are different formats in which you can download models, SafeTensors being of the most popular formats.

A safetensor file is made of 3 sections, header size, JSON header, tensor data. A quick aside; a tensor is a multi-dimensional array or numbers, where tensordata here represents the model weights.

The first 8 bytes together form one unsigned 64-bit integer, stored in little-endian order, and that number tells us how many bytes the JSON header takes up. The header itself is a group of key value pairs where each tensor's name points to information about that tensor, like dtype for the stored data type, shape for its dimensions, and data_offsets for where its raw bytes live. Those offsets are measured from the

start of the tensor-data section, the start is included and the end is one byte past the tensor, so subtracting them tells us how many bytes it occupies. There can be a __metadata__ entry as well, which we skip because it describes metadata rather than an actual tensor.



What we implement:

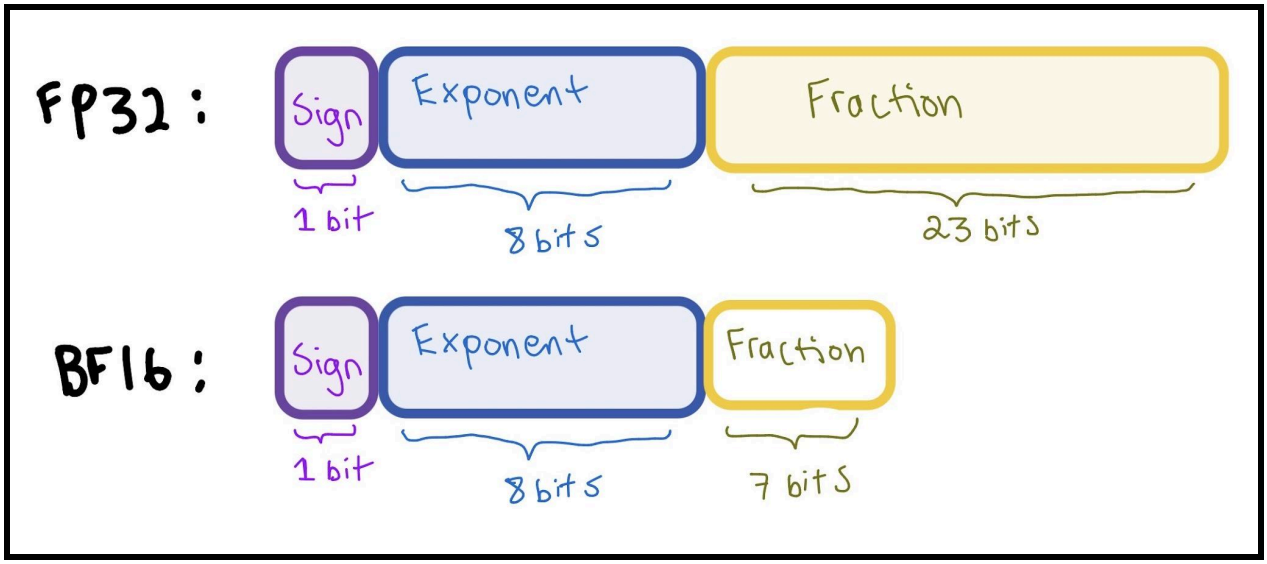
```
LlamaForCausalLM(
  (model): LlamaModel(
    (embed_tokens): Embedding(128256, 2048)
    (layers): ModuleList(
      (0-15): 16 x LlamaDecoderLayer(
        (self_attn): LlamaAttention(
          (q_proj): Linear(in_features=2048, out_features=2048, bias=False)
          (k_proj): Linear(in_features=2048, out_features=512, bias=False)
          (v_proj): Linear(in_features=2048, out_features=512, bias=False)
          (o_proj): Linear(in_features=2048, out_features=2048, bias=False)
        )
        (mlp): LlamaMLP(
          (gate_proj): Linear(in_features=2048, out_features=8192, bias=False)
          (up_proj): Linear(in_features=2048, out_features=8192, bias=False)
```

```
          (down_proj): Linear(in_features=8192, out_features=2048, bias=False)
          (act_fn): SiLUActivation()
        )
        (input_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
        (post_attention_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
      )
    )
    (norm): LlamaRMSNorm((2048,), eps=1e-05)
    (rotary_emb): LlamaRotaryEmbedding()
  )
  (lm_head): Linear(in_features=2048, out_features=128256, bias=False)
)
```

This is pretty much the entire model architecture for Llama 3.2 1B instruct. What does that mean? Pretty much just outlines the different operations we need to run, as well as the data shapes and types that we need to use.

So this gives us the model's components and dimensions, but we still need to know the order we run them in and what data type the weights use. We can search up the model on huggingface and we can find out that the dtype for the model weights are BF16

-> BF16 is a 16 bit floating point number, we use it because its a compact format that allows us to still access the full range of a 32 bit float, it is separated into 1 sign bit, 8 exponent bits, and 7 fraction bits, both a 32 bit float and 16 bit float have the same number of exponent bits but fp32 has more fraction bits (23).Meaning pretty much that the tradeoff of using this more compact format is a bit of precision. For inference this is a positive tradeoff, this small loss of precision is worth the benefit from effectively halving memory use.



Like I think I mentioned before, we need to know the order of operations to effectively produce correct output from the model, as the operations are different computations in certain order to produce a certain output.

For this we need to search up the LLM architecture, diagrams are good and Sebastian Raschka has a bunch of them here or just search it up. So lets go through the diagram and see what actually happens to the input..

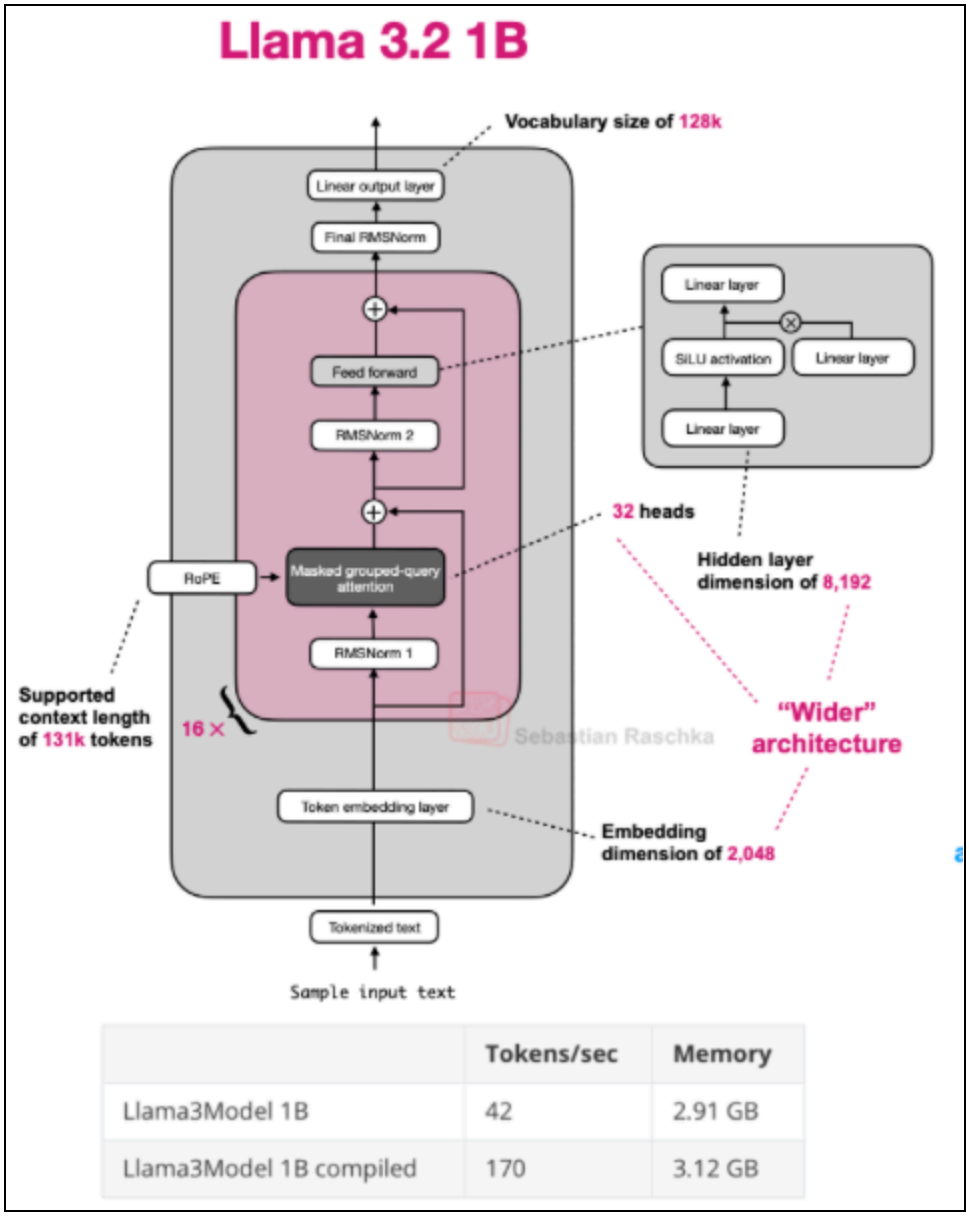


Figure 1: Llama 3.2 1B Architecture Diagram [*Sebastian Raschka*]

So we can see that we start off by sending some input text to the model. We then turn them into tokens, we do this using a Tokenizer , this converts text into pieces the model recognizes. So in this phase the tokenizer takes our input and splits the sentence or phrase into smaller pieces, which could be whole words, pieces of words, punctuation, etc. Then it assigns IDs to these specific pieces, it does this because it created a fixed list of token pieces during its training, each mapping to an ID, the ID value itself is arbitrary. You might think, okay well what if it wasnt trained on a certain input,

well like i mentioned before then the tokenizer would break it down as much as possible (split ex.: [“hyperglob”] -> [“hyper”, “gl”, “ob”]) until it resembles something part of the tokenizers vocabulary. Then we hit up the embedding table, which contains one learned starting vector for every token ID, and the ID we got from the previous step tells us which row to grab. The other model weights, like the Q/K/V and FFN matrices, are separate tensors (embed_tokens): Embedding(128256, 2048) This is the code that tells us that there are 128,256 possible token IDs and 2048 learned weights, which form an *Embedding Vector* . That pink box, is called a transformer block also called layers, there are 16 of them that have a shit ton of operations that you might not know yet but will get a clear understanding as we build more of the engine. They consist of these computations:

- RMS Norm
- Residual connection
- Masked grouped-query attention, which consists of:
    - Q projection
    - K projection
    - V projection
    - RoPE with Q projection
    - RoPE with K projection
    - Attention
    - Attention scores
    - Causal mask
    - Softmax
    - Residual connection
    - Attention scores with V projection
- O projection (output projection)
- Residual connection add
- RMS Norm
- Feed forward (like in first neural networks, Multilayer perceptron), which consists of:
    - Gate projection, first linear layer
    - Up projection, second linear layer
    - SiLU activation function, similar to ReLU but it's looking more like a sigmoid
    - Down projection, third linear layer
    - Residual connection add

An aside on memory:

Data can live in host or device. Host would be the actual PC using the CPU, it has slow big memory DRAM, DRAM is separate from the CPU but the CPU has its own very quick but small on chip memory called SRAM. This is where we have stuff like L1, L2 cache which allow for quicker access to recently accessed memory. Device would be the GPU, it also has slow big memory called VRAM, and also has quick small memory SRAM. Like I mentioned before we want to do the large computations on the GPU, so it needs access to the model weights, input tokens and intermediate results as well. In our engine we explicitly allocate device memory and copy the data from CPU RAM into GPU VRAM, so the kernels can repeatedly read it through the GPU's own high-bandwidth memory. CUDA does have ways for GPU code to access host memory, but keeping the data we repeatedly use on the device avoids going back across the CPU-GPU connection for those reads. This isn't as easy as it sounds at the low level though. On the cpu we have to create a variable and write to it, the compute the size of memory taken by the variable, i.e how many elements are stored, then multiply that by the size of the variable type and thus you know the total amount of bytes. Then on the gpu allocate memory so we have somewhere to put our stuff, then copy it over, now we can effectively use this data in GPU computations as its on our device VRAM. For the GPU memory allocation and copy there are CUDA specific functions which are quite similar to ones I worked with in C, like cudaMalloc, and cudaMemcpy.

Loading the Model:

It's time to start work and the first step when working with inference is to actually load the model you're working with. Remember how we talked about copying the weights from CPU RAM into GPU VRAM so the GPU can keep reading them efficiently, thats effectively what we're doing now, VRAM isnt necessarily bigger than CPU RAM, it's memory attached to the GPU and is a form of DRAM as well. Remember how before we said that a model is actually a single file with weights, thats pretty much what we're reading now, a model.safetensors file. Remember before that A safetensor file is made of 3 sections, header size, JSON header,

tensor data. Where tensor data is actually many tensors. So we have to read from safetensors file, to do this remember in C how we had to create actually open a file using fopen or open(), its the same thing in C++ but now we use std::ifstream, meaning input file stream which creates an object that lets cpp program read data from the file. We also have to use ios::binary to say the file is raw binary data so it preserves every byte exactly as if we dont sometimes the compiler tries to optimize or translate special chars or line endings while reading which would corrupt bytes and make offsets wrong.

Once we open the model file, we do in fact need to read it as well. As we've already mentioned numerous times, the file is a safetensor file. Which is formatted by the header, the JSON header which includes pretty much meta data on every single tensor in the model. There are 16 transformer layers, each of which have 9 different tensors inside of them. These 9 different tensors occupy some space in that file, these are recorded by their offsets, which can be found in the JSON header. This is important because at some point we're going to have to use these model weight values and we can't keep reading from the file. Super slow! We have to bring it to the GPU memory (VRAM) So how? Now we've opened the file which is the first step, but we also have to read it. Why? Because that can tell us a bit more about how we should be specifically parsing this. As I mentioned before each of the tensors are multidimensional arrays of floating point values that are weights for us. Each having a unique name as their key that defines what the tensor is actually for. Meaning if later on we need to find these tensors knowing the naming convention allows us to be able to quickly find what we're looking for. We'll get into that a bit more later, but thats just a preface on the motivation behind reading and moreso printing what we find from reading the JSON header that describes each tensor. So how do we read? Well first we have to read the original header, a safetensor file starts with the header size as a uint64_t meaning 8 byte value to tell us how large what we're reading next is going to be. Then we call .read(destination, bytes to read) on the safetensor file that we opened. It will return to us the value of the first 8 bytes as we had set it that way. Now we know how big the json header is so we can read it to completion. Now that we know we can use the .read again but now for the next headersize amount of bytes. One thing to mention, we are calling read again on our safetensor file, meaning the file pointer is currently at the start of

the json header anyways, i.e 8 bytes in so we don't have to handle for that. But what do we read to, read needs a destination buffer, of type char, an array of chars, which is a string. So we make a string of size headersize, initializing it to \0. Then we read headersize amount of bytes to it. Great! Now we have the JSON header, and we can parse it for the offsets that we will need for 2 things. 1) The max offset will tell us how much space the tensordata takes up, as remember we know how big the header is but not the raw tensor data i.e the weights. 2) It would allow us to read the starting offsets, which we need later to create pointers to them, as we will be allocating all this memory to the gpu, and we want pointers on our cpu memory so we can quickly access the gpu memory. Okay, so how do we go about parsing?

The way we go about parsing is: we've already created a string which has our JSON header in it. We use the JSON library that we imported before to convert our string, `header`, into a JSON object. We do that using the `json::parse` function.

As I mentioned before, we need to remember the starting offset so we can create our pointers later. The way we can go about that is using a hashmap, which in C++ is an `unordered_map`. The key-value type would be a string for the key, and a `uint64_t` for the value.

We also initialize our `max_offset` at zero because we will continuously check that throughout each iteration of a `for` loop that goes through the items of the header, which is now a JSON object.

We do this using a structured-binding `for` loop, which will go through the `name` and `tensor_info` for each JSON-header item. This is where having printed the header before would be useful, as you would know exactly what we are looking for when we need to find specific values from the JSON object.

For example, the data offsets are found at `"data_offsets"` in the JSON object. `data_offsets` itself is an array of two values: the start offset and the end offset. Those can be found by using `data_offsets.at(...)`, then converting them into a `uint64_t`, which is a 64-bit unsigned integer.

Then we use emplace. That essentially adds those values to our hashmap, and then we recompute max_offset by checking the current tensor's end offset against the current maximum offset.

Now we have a hashmap of our offsets which is good, but we still need to actually copy the full tensor data to memory so we can copy it to the GPU.

That part is actually relatively simple due to what we did with maxoffsets. Since the file ptr is already at the start of the raw tensor data and we know how large it is, we can just simply call read again.

Brings about the question though, where should we store this. We will use a vector of type char, as that is what .read expects but also it matches the file's exact raw byte layout. If you remembered from before yes we said the actual floating point values are BF16 which is 16 bits, meaning simply they are just two contiguous 8 bit chars.

Now all the raw tensor data is in memory, and we must copy it to the GPU but before that we need to allocate space on the GPU. Use cudaMalloc which will allocate a space on the gpu and set a pointer to point to the start of the GPU block reserved for this data. Hence before we allocate we need to initialize a void pointer as null. So it can be assigned a value. Then we use cudaMemcpy to actually run the memory copy. Has pretty simple params that we've already made so it works out nicely.

So initially, the way I went about is creating a hashmap of pointers each pointing to where in GPU memory that weight was. However, this would need string construction, hashing, and a hashmap lookup. Which is actually fine on initial run. As this is just first phase on loading, however when actually running inference we will have to go through all 16 layers of this model meaning if we were to go with this approach it would go through alot of string constructing and hashing to get the actual pointer which is slow. Instead by creating a struct,that has all the potential values for each layer and then indexing by that layer we get a much quicker lookup as well as no string construction when going through each layer.

Though that means we need to handle that construction on initial load instead. So think about it as we create BF16 type pointers to specific values, then the ones that repeat in every single transformer layer we create an array of 16 where each value is a BF16 pointer. To that layers specific tensor.

Then we loop through each layer and do the math, see where each pointer should point to and save it for that value in the struct so it can be called from cpu memory later on.

First though, we get the base by seeing where the memory points to in the gpu as we got that returned to us before from cudaMalloc and there is a pointer we can store it in, in the weights struct. Then we calculate the location by using the base and the offset hashmap from before and do this for each tensor in each layer.

By the way, we need to use weights outside of this function, so the way we did that was passing it by reference into the loading function itself.

The struct approach works because we are building this specifically for Llama 3.2 1B, and we know the model architecture, and we printed the JSON header to see how tensors were named.

Tokenization

We covered this briefly before but not to the max extent, it is pretty much the process of taking the input text, and splitting it into many parts, each of these parts receiving a token ID through tokenization. These token ids are what is used to look up the specific token in our embedding table, that row of values (vector) is the tokens starting representation. The transformer layers then use attention and other operations to update hidden state vectors based on the surrounding tokens, producing contextual meaning. Ok so when it comes to actually using a tokenizer often when a model is released it will come with a tokenizer already and or there are open source libraries that will provide them. So it starts off with getting an actual tokenizer file, this is given with the model itself. Think about it like this, you know how our LLama model was trained and its output was a simple safetensors file with a bunch of weights, thats the output of the tokenizers training with tokenizer model file. The tokenizer was

trained with byte pair encoding which pretty much means, pieces of text were split into UTF-8 bytes, each byte as a separate piece and then you repeatedly merge adjacent bytes that show up frequently in training data, then give each merged piece a token ID. Anyways, now we have all these pieces of text and their token ID values, and this is what gets stored in the tokenizer file. The tokenizer vocabulary is basically the tokenizer's dictionary: it contains every text/byte piece the tokenizer knows about, along with the fixed token ID assigned to that piece.

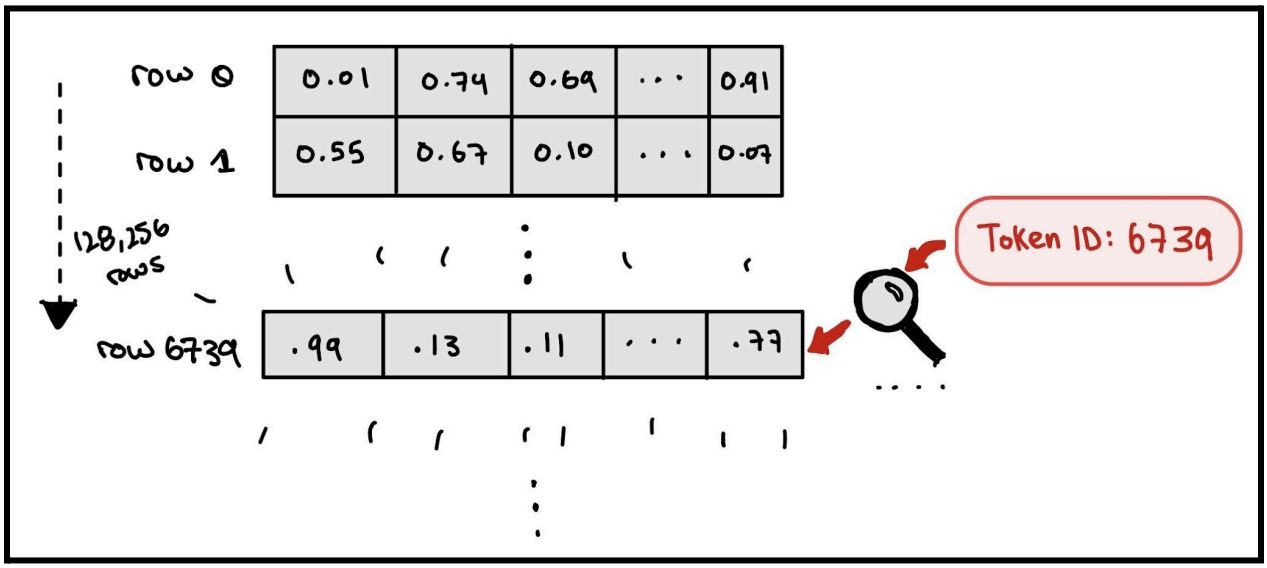
So when we actually give the model a prompt like "Hello world", the tokenizer library first looks at the text and splits it up based on its rules, like spaces, punctuation, words, numbers and things like that. It then turns those pieces into UTF-8 bytes. UTF-8 is just the standard way computers represent text as numbers from 0 to 255 called bytes, so it can handle normal English text, punctuation, emojis and other languages all using the same system.

It then starts checking which byte combinations exist in the tokenizer vocabulary. It will keep merging byte pieces together based on the BPE rules it learned during tokenizer training. So instead of every character always being its own token, common things like " hello", "ing", "tion", or even whole words might already exist as one token. Once it has found the final pieces, it looks up the token IDs that were assigned to them when the tokenizer was trained.

One thing to note, as you'll see we are using an instruct model, but what does that mean? It just means our actual prompt also has a chat format around the words we typed. So our tokenizer adds the beginning-of-text token, a user header, the message and end-of-turn token, then starts an assistant header so the model knows whose text to continue. These are extra token IDs that go through the engine too and take up context positions just like the message tokens.

Anyways so the final result is just a list of integers, something like [128000, 9906, 1917]. The tokenizer does not actually do any neural network calculations itself; it is really just translating text into the integer IDs that the trained Llama model understands.

Then our C++ inference engine takes those IDs and copies them to the GPU. From there, the embedding lookup uses each ID to grab the correct row from the embedding table in our safetensors weights. Those embedding vectors become the starting hidden states that go through all of the Transformer layers.



At the end, the model gives us a new token ID that it thinks should come next. We then use the tokenizer in reverse, where it looks up that ID in the vocabulary and turns its stored bytes back into readable text.

input text
→ tokenize → token IDs → embedding lookup → Transformer layers
→ next token ID → tokenizer decode → output text

Normilization using RMSNorm:

As we saw from the diagram above (Figure 1) the next phase of our inference engine is for our tokens to go through RMSNorm1 which means the first Root Mean Square normilization, furthermore it's happening inside of a Transformer layer, meaning its a repeated calculation for each layer. What is it/ Why do we do it, so firstly the mathematical operation of RMS Normilization is pretty much just dividing the vector by its root mean square, meaning before the learned scale its

RMS comes out close to 1, the ordinary average doesnt have to be 1 and individual values can still be negative or much bigger than 1. In our model the full calculation is

$$y[i] = x[i] / \text{sqrt}(\text{mean}(x^2) + 1e\text{-}5) * \text{gamma}[i]$$

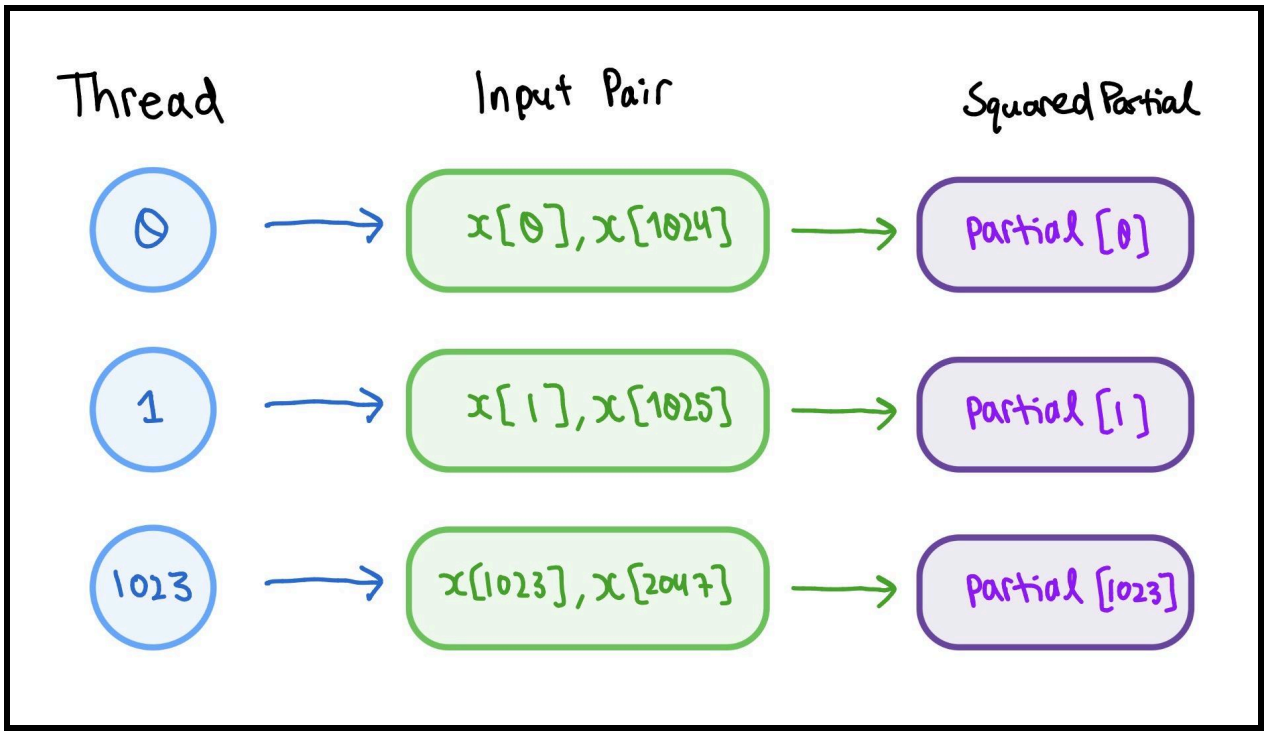
where gamma is a learned weight for each of the 2048 coordinates, so that final multiplication can change the direction too, and dividing by the RMS can make a small vector bigger just as it can make a big one smaller. Why do we do this, and doesn't this affect our weights? As I mentioned earlier, how we look at the vectors from the embedding table is that their semantic value is embedded in its direction. A practical example would be if say the word woman, man, brother, and sister each had its own token ID in our vocabulary. If you were to take brother, subtract the man vector, then add the woman vector, the vector would point in the direction of sister. All this while saying the actual magnitude didn't matter, just the direction. So if we were to normalize this, meaning the relation between the values in the vector would be the same but their actual magnitude would be less. The actual calculation is very simple. We just take each value in the vector, square it, then get the average, then get the square root of this average. Lastly divide each value in the vector by by the RMS value.

Ex. [1,2,3,4,5] -> [1,4,9,16,25] -> $\mu = 11$ -> $\sqrt{\mu} = 3.316$
-> [0.302, 0.603, 0.905, 1.206,1 .507].

We went over how it works, why it works, but why do we do it? Well as the hidden states go through all these operations their scale can change, and normalization helps keep that scale manageable, which also helps when training the model. Remember though during inference we're changing the hidden states, the learned weights themselves stay fixed. You may have heard of LayerNorm as well, it subtracts the mean and scales using the standard deviation, whereas RMSNorm scales using the root mean square without subtracting the mean, and both use learned scale parameters. Researchers found RMSNorm could give comparable performance in their experiments while doing less computation, and our Llama checkpoint was trained with it, so we need to reproduce that same operation during inference as well, we can't just swap in another normalization and expect the same outputs.

Understanding the code:

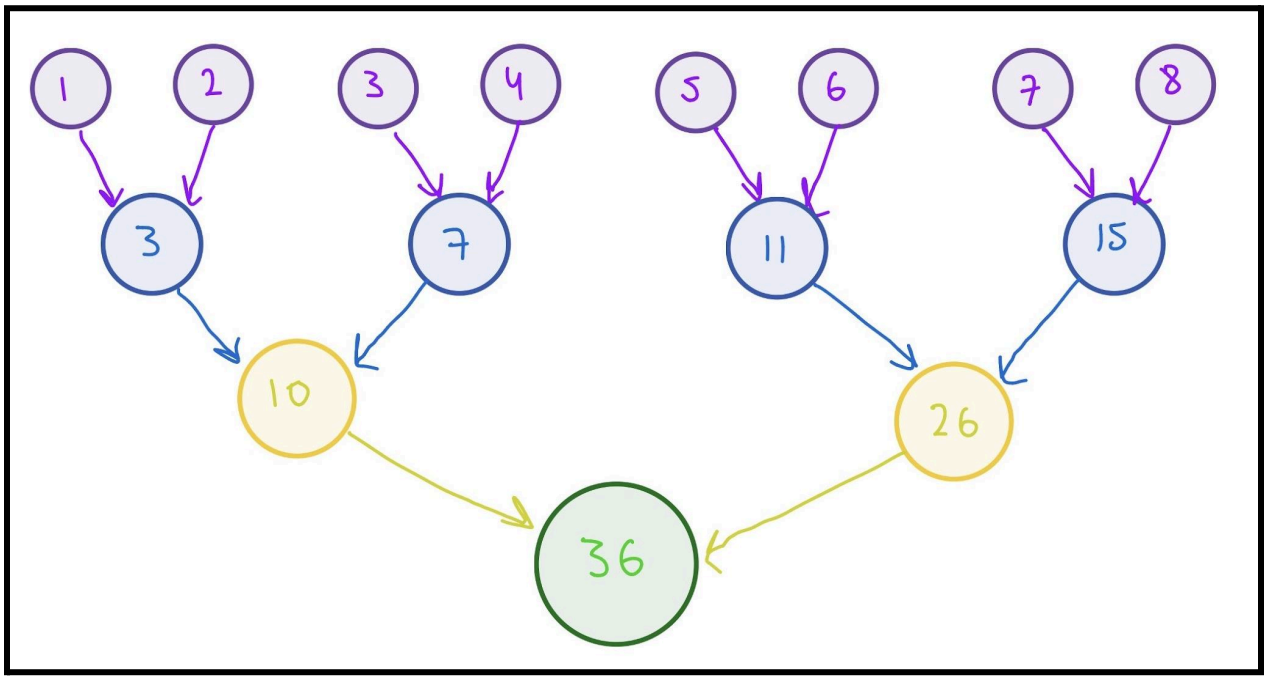
RMSNorm treats each token IDs 2048-value hidden vector (vector that comes initially from the embedding table, These are called hidden values because they are the model's internal representation rather than directly interpretable outputs.) independently. Before we do anything, as we did when loading the model. Computations are to be done on the GPU so we have to allocate and copy over the actual vector of token IDs to the GPU using cudaMalloc and cudaMemCpy. Then we are going to have to deal with a table of all the token IDs and their respective hidden values, as afterall these are the weights we are going to be working with. And so you allocate space on the GPU for that too, you dont re-create it on cpu to copy on to the GPU though, as if you remember correctly our preliminary work to put all the weights on the GPU as well as create a pointer directly to the embedding table was already done, no need to duplicate efforts. Now we're going to be writing a CUDA kernel, which is pretty much just a way of saying a function that will run on the GPU. But instead of running just once, the GPU is going to take this exact same function and run it thousands of times simultaneously across all of its cores. Taking complete advantage of parallelism. __global__ declares a CUDA kernel. It runs on the GPU and is launched by CPU code using <<<blocks, threads>>>. Passed into our kernel is a ptr to the input tensor with token_count rows, and 2048 columns. As well as an identically sized output tensor for normalized values. Lastly a weight vector containing 2048 weights, each token uses same weight vector. In this kernel, we face an interesting parallelism problem, where the threads actually have to communicate with each other. In this case its due to the fact that you require a total sum for one of the steps of the computation of RMSNormilzation vector. One thing about CUDA kernels is that you can only at once spawn 1024 threads in a block, this is important as we can only safely share partial sums amongst threads within the same block. Meaning 2 blocks won't work so the solution would be to have each thread handle two of the token's 2048 hidden values and stores their squared sum in its spot.
Ex: `partial[tid] = x0 * x0 + x1 * x1;`



Which brings us to __shared__ which pretty much just means shared memory, that any of the threads in the block can read/write to. It is in float instead of bf16 as here with all these computations we want to assure as much precision as possible before its lost later on, if we compute using bf16 now it'd be farther away from the true value. One block handles one token. It stores each threads partial sum. The example illustrated above is not necessarily how we take the value though. We do it by taking it from the first half squaring it, and adding from the second half squaring it and putting that value in the partial sum vector. So it'd be 1024 away from base + tid. hen we use __syncthreads(), which is a barrier for the threads in this block, so each thread waits until the others reach that point as well before continuing. This means we can now read the partial sums knowing the other threads have finished writing them, the threads keep running afterwards, unlike pthread_join() which you may be familiar with which waits for a thread to finish completely. We want to find the total sum of these squared values so we can calculate the RMS, and adding them one by one would take more sequential steps than we need. So we use a tree reduction, like instead of one person adding a list of 8 numbers with 7 additions in a row, multiple threads add pairs at the same time, then add the results in pairs again until one total remains. Ex:

- **Step 1:** 4 processors add 4 pairs of numbers at the same time. (You are left with 4 results).
- **Step 2:** 2 processors add those 4 results in pairs. (You are left with 2 results).
- **Step 3:** 1 processor adds the final pair to get the total.

The Result: It only took 3 steps instead of 7.



In our case we would have 512 → 256 → 128 → 64 → 32 → 16 → 8 → 4 → 2 → 1. So in the first round we'd have:

```
partial[0] += partial[512]

partial[1] += partial[513]

...

partial[511] += partial[1023]
```

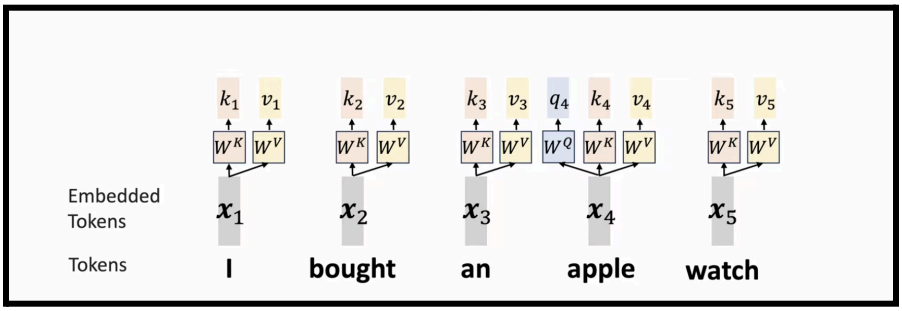
Now, there are 512 useful sums, then we keep breaking it down until the total sum remains in partial[0]. We then join the threads again, but why? It works as a barrier, this is a mechanism in concurrent programming to ensure one thread does not move on to the next iteration of the loop before the first one is complete. Ensuring correct ordering of data read.

RoPE (Rotary Positional Embedding/Encoding):

The motivation behind this comes from the fact that a transformer is unable to tell the difference in the meanings of words by their position in a given statement. An example of this could be:

"I bought an Apple watch" and "Watch an Apple I bought"

In this example the input is made up of all of the same tokens except in a different order, does that matter? Yes, because in this case apple refers to two different things here, but if Apple itself has one embedding vector it is true that the vector is the same regardless of where in the input prompt it lives. Lets do a short run-through of what exactly happens during the transformer layer. Each token is multiplied by Query, Key, and Value matricies to output their respective Q,K,V vectors for that embedding token.

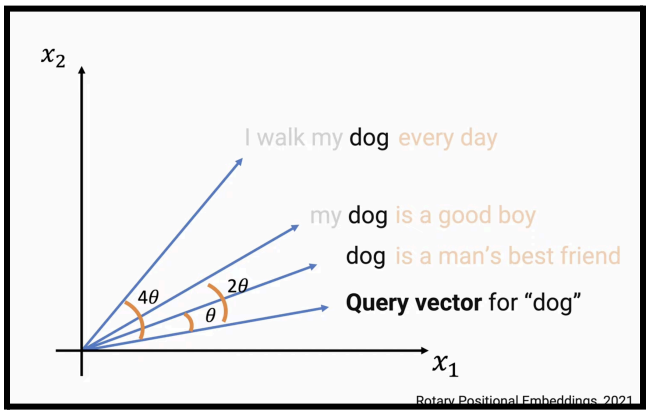


Looking at apple as being x4, with q,k,v vectors q4,k4,v4 respectivley. You get the attention vector for apple by taking the dot product of q4 and k^T then divide by sqrt(d_k) and apply softmax, just as we saw in the attention formula, just this time for one vector instead. Lets say, we did not use masking here, and we were computing for an output vector after multiplying by value vectors, if we were to shuffle the words around as we did earlier with the example, then the weighted sum would be the exact same, meaning the transformer would see both sentences as the exact same despite them meaning totally different things. Original transformer paper introduced a sinusoidal positional encoding method where each position is assigned a unique vector of periodic components with varying frequencies. We then need to inject this positional information into the embedding vectors, and

there are multiple ways of doing it, like concatonating them, adding them, or use MLP. Original paper uses adding.

With absolute positional encoding we give each token information about where it is in the sequence, but having different positions doesnt automatically mean two sentences have different meanings, the model still learns how to use that information. With RoPE the positional part of the dot product between a query and key depends on the difference between their positions, so if we shift both positions by the same amount that difference stays the same, which is what makes this relative. If we actually rearrange the words though we can change their relative distances and order, and we still want the model to notice those changes, so RoPE doesnt mean we can shuffle a sentence around and expect the attention or meaning to stay the same.

Now lets finally get more into RoPE. For simplicity of understanding imagine a two dimensional query vector for the token dog, RoPE rotates this vector in R^2 by θ depending on its position in the input. It only depends on the position, meaning we do not care how many or what tokens come after it when we are encoding.



Lets look an example of RoPE. In Transformer models, attention between words is calculated using the dot product of a Query vector (the word looking for context) and a Key vector (the word providing context). Lets say a sentence has ‘I’ and ‘dog’ where dog is the query vector and I is the key vector. In this case that means wes are looking at how much attention the token ‘dog’ pays to the token ‘I’. Let’s look at the sentence “I walk my dog”, where I is at pos 1, and dog is at pos 4. Meaning their key and query vectors are rotated by 1θ and 4θ

respectivley. How are they rotated, well, they are multiplied by this matrix. ->

$$q_{\text{"dog"}} = \begin{bmatrix} \cos(4\theta) & -\sin(4\theta) \\ \sin(4\theta) & \cos(4\theta) \end{bmatrix} W_q x_{\text{"dog"}}$$

Query vector for "dog"

Then we take the dot product of the query and key we've already rotated, so in the example above their rotations are separated by 4θ - 1θ = 3θ, and that relationship shows up automatically in the result. The dot product is one number, we arent rotating it afterwards. So if we shift both positions forward by the same amount, their individual rotation angles change but the difference stays the same, which keeps the positional part of their comparison the same as well.

Now that we've seen an example, we're ready for a general formulation, where

$$R_\Theta^m = \begin{bmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{bmatrix}$$

Here, m is the query's position in the input sequence, and our code starts counting positions at 0, so during prefill the bounds are 0 <= m < token_count, where m is an integer. We rotate the query at position m and the key at position n before taking their dot product, and because transposing a rotation reverses its angle, that comparison can be written in terms of the difference between their positions. During decode the position keeps counting through the whole sequence, so like we explain later, row 0 of the current input buffer doesnt mean position 0 in the sequence.

$$\alpha_{m,n} = x_n^\top W_k^\top \quad R_\Theta^{m-n} \quad W_q x_m$$

This simplified example only works in the 2D case though, in the case of higher dimensions we split them into groups of smaller ones with rotanial encodings then combine after. But this takes alot of memoery, so we can do element wise operations. Note: lots of examples coming ->

Here is one tiny RoPE example with one 2-number pair. Real Llama heads have 32 pairs, but the same calculation happens for each one.

Assume the query for token A is:

$$Q_A = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

Assume the key for token B is:

$$K_B = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

Let their positions be: $p_A = 2$, $p_B = 5$

And, just for this example, let this pair's RoPE frequency be: $\omega = 0.1$

In reality we use the frequency formula we see in the code for each pair, RoPE calculates one angle per token:

$$\theta_A = p_A \times \omega = 2 \times 0.1 = 0.2$$
$$\theta_B = p_B \times \omega = 5 \times 0.1 = 0.5$$

Now rotate the query:

$$Q'_A = \begin{bmatrix} 2 \cdot \cos(0.2) - 1 \cdot \sin(0.2) \\ 2 \cdot \sin(0.2) + 1 \cdot \cos(0.2) \end{bmatrix}$$

Using: $\cos(0.2) \approx 0.9801$, $\sin(0.2) \approx 0.1987$, we get:

$$Q'_A = \begin{bmatrix} 2(0.9801) - 1(0.1987) \\ 2(0.1987) + 1(0.9801) \end{bmatrix} = \begin{bmatrix} 1.7615 \\ 1.3774 \end{bmatrix}$$

Now rotate the key:

$$K'_B = \begin{bmatrix} 3 \cdot \cos(0.5) - 4 \cdot \sin(0.5) \\ 3 \cdot \sin(0.5) + 4 \cdot \cos(0.5) \end{bmatrix}$$

Using: $\cos(0.5) \approx 0.8776$, $\sin(0.5) \approx 0.4794$, we get:

$$K'_B = \begin{bmatrix} 3(0.8776) - 4(0.4794) \\ 3(0.4794) + 4(0.8776) \end{bmatrix} = \begin{bmatrix} 0.7150 \\ 4.9486 \end{bmatrix}$$

Attention now compares the rotated query and key with a dot product:

$$Q'_A \cdot K'_B = (1.7615)(0.7150) + (1.3774)(4.9486) \approx 8.075$$

That 8.075 is this pair's contribution to the attention score. The important part is that token A was rotated by 0.2 radians and token B by 0.5 radians. Their difference is:

$$\theta_B - \theta_A = 0.5 - 0.2 = 0.3$$

which is also: $(p_B - p_A)\omega = (5 - 2)(0.1) = 0.3$

So the comparison naturally contains their relative distance:

$$p_B - p_A = 3$$

In a real head, this happens for all 32 RoPE pairs, and the attention dot product adds all of their contributions together.

So to put it simply all we're doing is rotating each weight in our query and key vectors for each of our tokens, they rotated based on which token position they have in the prompt, and then finally as you may be thinking dont we need to get the relation between all of them, yes we do that when we finally compute the dot product and the final attention scores go into its respective matrix, happens during attention.

How it works for each of the rotations within the vector, query OR key itself is as such:

For one token, the flow is:

$$token\ embedding \rightarrow Q,\ K,\ V$$

Take that token's Q vector. One head has 64 numbers:

$$Q = [q_0, q_1, \ldots, q_{31}, q_{32}, q_{33}, \ldots, q_{63}]$$

RoPE makes 32 coordinate pairs inside that one head:

$$(q_0, q_{32}),\ (q_1, q_{33}), \ldots\ ,\ (q_{31}, q_{63})$$

Then it rotates every one of those pairs using that token's position.

So for token 3:

$$(q_0, q_{32}) \rightarrow rotateusingposition3$$

$$(q_1, q_{33}) \rightarrow rotateusingposition3$$

…

$$(q_{31}, q_{63}) \rightarrow rotateusingposition3$$

That gives one finished position-aware query vector:

$$Q'_3$$

The same happens to its key vector:

$$K_3 \rightarrow K'_3$$

Only **after every token has its own rotated** Q **and** K does attention compare tokens:

$$Q'_3 \cdot K'_1$$

$$Q'_3 \cdot K'_2$$

$$Q'_3 \cdot K'_3$$

More examples just to drive home the concept:
Tiny fake head with 4 numbers instead of Llama's real 64:

$$Q_3 = [2,\ 5,\ 3,\ 7]$$

Using the same half-split layout as your code, its RoPE pairs are:

$$pair0 = (2, 3)$$

$$pair1 = (5, 7)$$

Now give each pair a different frequency:

$$\omega_0 = 0.1$$

$$\omega_1 = 0.01$$

Both pairs are from token position 3, so:

$$\theta_{3,0} = 3 \times 0.1 = 0.3$$

$$\theta_{3,1} = 3 \times 0.01 = 0.03$$

Pair 0 rotates more because its frequency is faster:

$$(2,3) \rightarrow \begin{bmatrix} 2 \cdot \cos(0.3) - 3 \cdot \sin(0.3) \\ 2 \cdot \sin(0.3) + 3 \cdot \cos(0.3) \end{bmatrix} \approx \begin{bmatrix} 1.024 \\ 3.457 \end{bmatrix}$$

Pair 1 rotates only a little because its frequency is slower:

$$(5,7) \rightarrow \begin{bmatrix} 5 \cdot \cos(0.03) - 7 \cdot \sin(0.03) \\ 5 \cdot \sin(0.03) + 7 \cdot \cos(0.03) \end{bmatrix} \approx \begin{bmatrix} 4.788 \\ 7.147 \end{bmatrix}$$

So the rotated query for token 3 becomes approximately:

$$Q'_3 = [1.024,\ 4.788,\ 3.457,\ 7.147]$$

Same token position; different pair frequencies; different rotation amounts.

In the real model, this happens for all 32 pairs in a head.

Remember though this is how the frequencies are actually calculated:

$$\omega_i = \frac{1}{\theta^{2i/d}}$$

$$\theta = 500000$$

$$d = 64$$

Theta and d values coming from the LLama model configuration.

Last thing to note, is that in LLama we also scale the frequencies with pairs, why and how we do this noted below:

Each RoPE pair has a frequency ω_i, which determines how quickly it rotates as token position increases.

$$angle_{p,i} = p \cdot \omega_i$$

A pair's wavelength is the number of token positions needed for one full rotation:

$$\lambda_i = \frac{2\pi}{\omega_i}$$

Therefore:

$$large\ \omega_i \Rightarrow short\ wavelength \Rightarrow fast\ rotation$$

$$small\ \omega_i \Rightarrow long\ wavelength \Rightarrow slow\ rotation$$

Llama divides RoPE pairs into three wavelength regions:

$$\lambda_i < 2048$$

Keep fast frequencies unchanged:

$$\omega'_i = \omega_i$$

These pairs preserve detailed positional differences between nearby tokens.

$$2048 \leq \lambda_i \leq 8192$$

Smoothly blend between original and slowed frequency:

$$s = \frac{8192/\lambda_i - 1}{4-1}$$

$$\omega'_i = (1 - s)\left(\frac{\omega_i}{32}\right) + s\omega_i$$

This prevents an abrupt change between neighboring pairs.

$$\lambda_i > 8192$$

Slow the already-slow frequency:

$$\omega'_i = \frac{\omega_i}{32}$$

This stretches its wavelength:

$$\lambda'_i = 32\lambda_i$$

Why: fast pairs retain precise local position information, while stretched slow pairs remain useful across much longer contexts.

Attention:

Single Headed Attention:

So this is the big kicker, and what you've probably heard a million times, especially on the famous paper 'Attention is all you Need' that really kickstarted the whole AI boom. In our engine we use Multi-headed attention but its good to start here to get a foundation.

Pretty much though, attention lets a token use information from other token positions to build up context. A good example could be 'I just clean windows' and 'I just downloaded windows', for us humans the words clean and downloaded make it pretty easy to tell which windows we mean. If we took each token's starting embedding and passed it through a feed forward network independently, then windows would start

with the same representation in both cases and wouldnt have access to those surrounding words, so that independent processing wouldnt give it the context we need. A feed forward network can use context if we actually supply it in the input, but here attention is the operation that mixes information between positions, meaning the representation for windows can change based on the words around it, but how?

The way we can measure the actual relevance from token to token would be through a vector dot product, you are already familiar with how we get token embedding vectors from the learned embedding table, and that's now a key part of what we do here? If you're keen you may have thought, why don't we just do the dot product of one token, by the other tokens to measure their similarity, the problem is often words that might highly relevant to each other end up having very different token embeddings, furthermore it can be asymmetrical meaning one word can mean alot more to another word then the other way around, an easy way to think of this is an adjective and a noun: **'Amazing food'** in this case their token embeddings would probably be very different leading to low similarity, as well as the fact that for **food**, the word **Amazing** is very relevant, but for **Amazing** it doesn't really care about the word food.

Rather than compare token embeddings directly, the model first learns two different representations of every token: a query and a key. The query represents what the current token is looking for, while the key represents what information that token can offer. We compute these values by multiplying a tokens embedding vector by a Query (W_q) and Key (W_k) matrix. W_q learns which combinations of a token's current features describe what it needs to find. W_k learns which combinations describe what that token can provide. Their dot product measures whether what one token seeks matches what another token offers.

Then, we simply take our token embedding vector and multiply it by these matrices to give us new vectors to use to calculate how much 'attention' one token pays to another. Multiplying it by those matrices can be called Q,K,V projection and it's what we do in cuBLAS, in Video 6 of the series!

So mathematically we want $Y = X \times W^T$, but there's a memory layout detail here as well, our buffers are stored row-major and cublasGemmEx

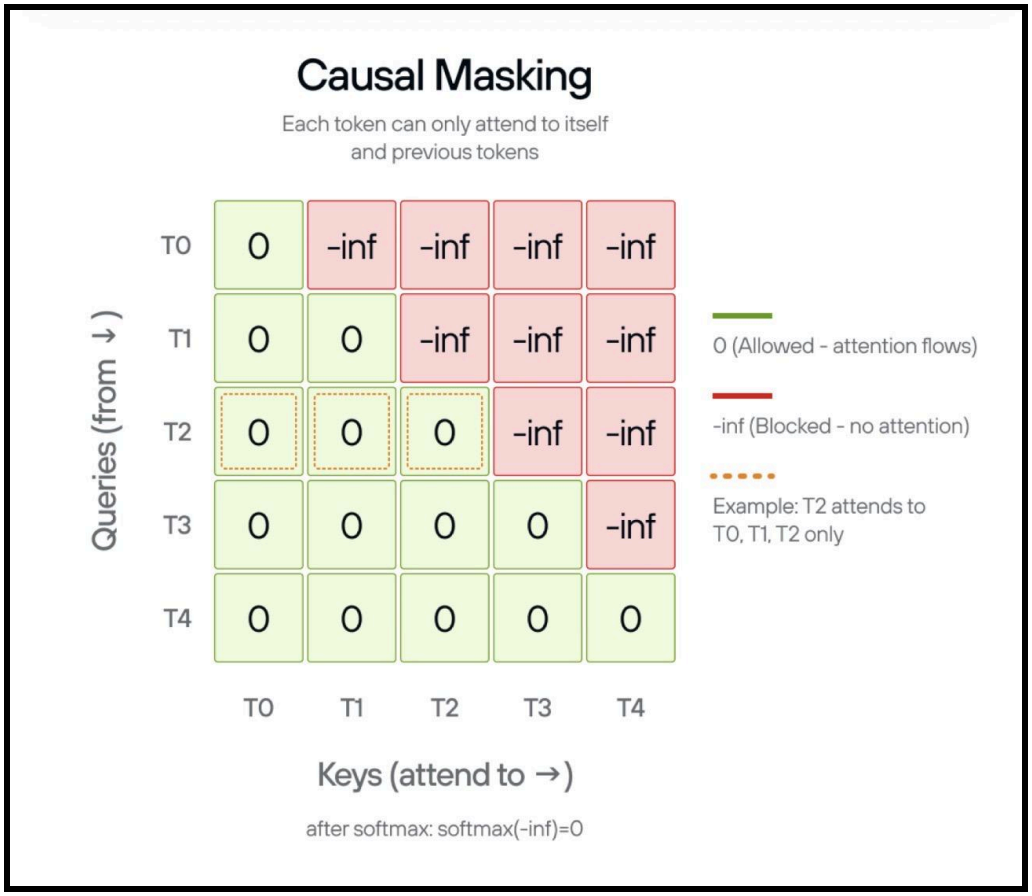
reads them as column-major. The same bytes of a row-major matrix look like its transpose when read column-major, so we arrange the call to calculate $Y^T = W \times X^T$ without physically moving the numbers into a transposed buffer. In projectBf16RowMajor we pass the weight pointer first with CUBLAS_OP_T and the input pointer with CUBLAS_OP_N, using m = output_features, n = token_count and k = input_features. The leading dimensions tell cuBLAS the stride through the stored matrices, so here lda and ldb are input_features and ldc is output_features. So if we later take one 64-value head out of a wider Q or K row, the row stride is still the full stored width, the head slice doesnt jsut magically become a tightly packed matrix.

Perfect, now you can think of their attention scores as being the dot product of the query and key vectors. The full attention score matrix can thus be given as

$$\boldsymbol{S = QK^T + M}$$

Where d_k is the number of values in one key head, so we divide the dot products by sqrt(d_k) to control how spread out the scores get as that dimension grows. We then add M, which is 0 for positions the query can attend to and -∞ for future positions, because we dont want a token looking ahead. The dot products can be negative as well, thats fine though because they’re scores and we care about how each one compares to the others. Softmax runs across the key positions separately for each query, we subtract that row’s maximum for numerical stability, take the exponentials and divide by their sum, so the resulting attention weights add to 1 and future positions get weight 0. Thus we are given the attention-weight matrix:

$$A = softmax(\frac{QK^T}{\sqrt{d_k}} + Mask)$$



The attention-weight matrix tells us **where** each token should look, but it doesn't yet tell us what information to take from those tokens. That is the purpose of the value vectors. Each value vector is created from the token's current representation using its own learned matrix: $V = X \times Wv^T$, you can think of values as the actual information that token can provide. Q and K determine the attention weights; V contains the information we transfer once we know which tokens matter.

We therefore multiply the attention-weight matrix by the value matrix:

$$O = A \times V$$

For a token i, its output is a weighted sum of the value vectors of the tokens it attends to:

$$o_i = A[i,1] \times v_1 + A[i,2] \times v_2 + \ldots + A[i,i] \times v_i$$

Or... $o_i = \Sigma\square\ A[i,j] \times v_j$

Or... o_i = sum over allowed j of A[i,j] × v_j

A high attention weight means that token's value vector contributes more information to the new representation of token i. The output O is therefore a new context-aware representation for every token.

Multi-headed Attention:

So now that we understand single headed attention we know that one head can look at all the earlier tokens to work out which ones are relevant, and then take a weighted mixture of their value vectors. But then, why not just use one massive attention head and call it a day?

The problem is that one attention head has to use a single definition of what "relevant" means. In a sentence, a token might need to care about multiple things at once, like maybe one earlier word tells us the subject, another tells us the tense, whereas another explains what a pronoun refers to, and another provides the actual topic. Having one head do all of this would be like asking one person to look for every possible pattern at the same time.

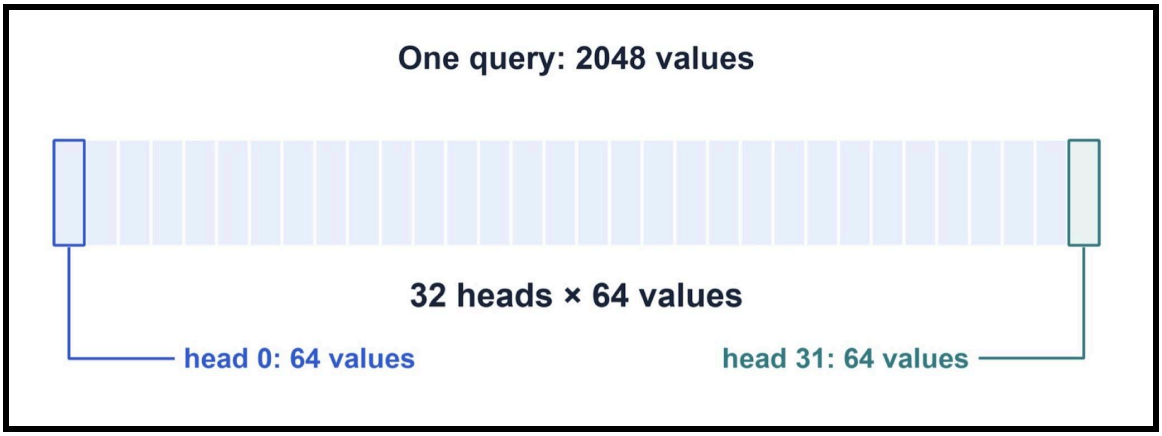
Multi-headed attention lets the model look at the prompt from multiple different perspectives at the same time. Rather than creating one query, key, and value vector, we just split them into multiple smaller q,k,v vectors, with one set for each head. Each head has its own learned section of the Q, K, and V weight matrices, so each can learn different useful patterns.

For our Llama 3.2 1B model, the query projection has 2,048 values per token. We split this into: 32 query heads × 64 values per head = 2,048

So, after the Q projection, we can think of it as:

$$Q = [Q_0, Q_1, Q_2, \ldots, Q_{31}]$$

where every Q_i is a smaller query matrix with shape: [# of tokens, 64].



The same idea then also applies to keys and values. Each head runs the exact single-head attention calculation independently:

$$S_h = (Q_h \times K_h^T) / \sqrt{64} + M$$

$$A_h = \text{softmax}(S_h)$$

$$\text{head_output}_h = A_h \times V_h$$

Something to keep in mind here is that with N prompt tokens each head has an [N,N] score matrix, because we compare every query with every key before masking the future positions. We have 32 query heads and store the scores in BF16, so if we allocate for 2048 prompt tokens that would be $32 \times 2048 \times 2048 \times 2$ bytes = 256 MiB just for this score buffer, which we reuse between layers. So doubling the prompt capacity makes this buffer four times as big, while the KV data for twice as many processed tokens only doubles. Our code sizes this allocation using the prefill capacity, and paging the KV cache later doesnt get rid of this dense prefill buffer, but thats for later ;)

Each head can now attend to different earlier tokens! One of the heads could learn that the **food** token should look at **Amazing** as a key, another could care more about the earlier verb, surrounding sentence structure, or something else. The heads aren’t manually assigned jobs, instead the model just learns what is useful during training.

Once every head has created its output, we concatenate them. This just means placing their 64-value output vectors beside each other.

$$\text{head_output}_0\text{: 64 values}$$

head_output_1: 64 values

...

head_output_{31}: 64 values

32 × 64 = 2,048 values

Meaning now for a layer, we have a 2048 valued multi head attention output per token We then multiply it by another learned weight matrix, W_o, called the output projection:

$$\text{attention_update} = \text{Concat}(\text{head_output}_0, \ldots, \text{head_output}_{31}) \times W_o^T$$

W_o lets the model mix the useful information discovered by the different heads. Finally, we add this attention update back to the token's previous hidden state using a residual connection, you can think of the attention update as being Δx and we are adding the change to the old embedding token to get new hidden state

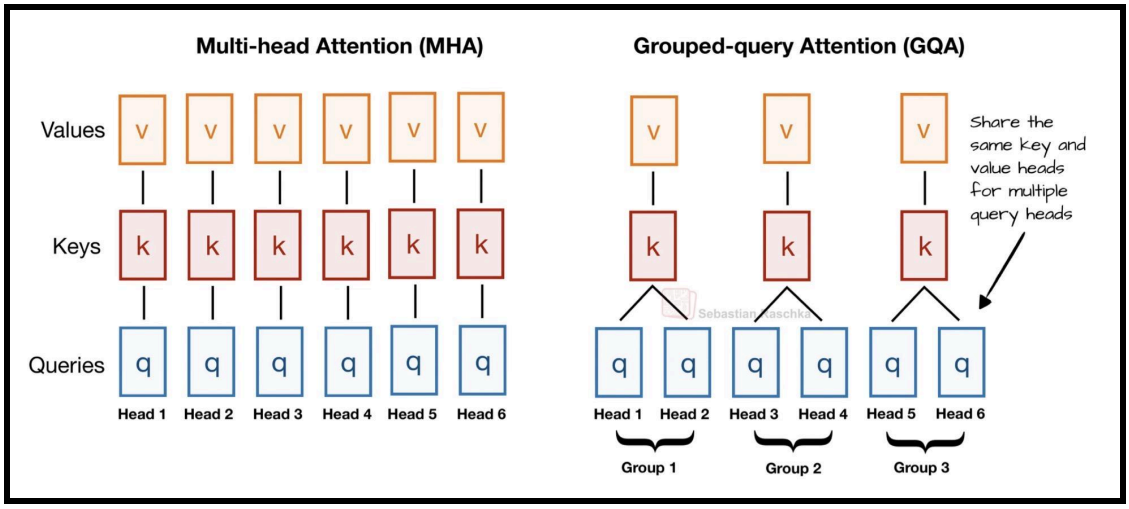
new_hidden_state = old_hidden_state + attention_update

Or... $x' = x + \Delta x$

Grouped Query Attention:

One interesting part of LLama's model architecture is that it uses Grouped Query attention, which pretty much means it splits key and value heads amongst multiple query heads. In our LLama 3.2 1B case in specific it has 32 query heads, but only 8 key heads and 8 value heads. This means every group of four query heads shares one key head and one value head:

Q heads 0–3 use K,V head 0
Q heads 4–7 use K,V head 1
...
Q heads 28–31 use K,V head 7



Here the picture uses groups of 2 query heads, our Llama model uses groups of 4.

Source: Sebastian Raschka

So now our actual calculation for each query head h would be:

$$kv_head = h / 4$$
$$S_h = (Q_h \times K_kv_head^T) / \sqrt{64} + M$$
$$A_h = softmax(S_h)$$
$$head_output_h = A_h \times V_kv_head$$

The reason we do this is that it keeps the multiple query perspectives that make multi-head attention so powerful, while using fewer key and value vectors. This saves a ton of KV-cache memory during inference. We'll get into why that's so important later on ;)

Feed Forward Network:

Ok perfect, so up until now, within the full model architecture of LLama, we've completed the attention part of one individual layer, you can call the first one layer 0. What does that mean though what have we gained from that, well now each token has an updated representation that incorporates information from itself and earlier tokens in the prompt, pretty much building context for the prompt and we see how that all worked with the mathematical computations behind attention.

Now as you can see in the architecture diagram there is a second RMSNorm, and what we pass into it is the updated hidden-state matrix

after the attention residual add. Remember W_o is the learned output-projection matrix, we used it to calculate the attention update, but the values we're normalizing now are x + attention_update. So we take those updated token vectors, apply this layer's second RMSNorm and pass the normalized result into the Feed Forward Network, (FFN).

Now the weights in the FFN are very important, they were maticulously modified during training to ensure that with the help of attention as well when we produced the next token and the ones after that it was a coherent correct answer.

Lets think through this with an analogy, think of a group of people working through problem, where where each person represents one token and holds a page of notes which is the token's vector.

> Attention is like exchanging information between the people, remember though not everyone can share due to causal masking, you can only share amongst the people before you.
>
> FFN is like after you have exchanged information with whoever you can for one round, everyone is given a paper of rules or hints that they apply to their answers individually without consulting anyone that further enriches this answer.

Within our model architecture, this process repeats 16 times between transformer layers.

FFN pretty much is just adding more information to our output vectors, independently, without the context of the other tokens within the prompt. It is looking at the features within the token, and using learned weights to build the representation better.

I hope that little intro made a bit of sense, but now we are going to go into the thing that really builds what a feed forward network really is. The perceptron, you can think of a perceptron as being one weighted decision, This means there are weights, and there are inputs and we apply certain weights to certain input values to compute a mathematical value that could bring us to some conclusion. This is best understood in an example.

Imagine a program that tries to decide whether a message is urgent. We give it two numbers:

x[0] = 1 if the message contains “urgent”; otherwise 0
x[1] = the number of exclamation marks

And then we have a code representation of a perceptron:

```
float score =
    1.5f * x[0] +
    0.25f * x[1] -
    1.0f;

bool is_urgent = score > 0.0f;
```

You can see the weights we applied, we put a more significant importance to the number of times the message contains urgent as opposed to the number of exclamation marks, and this makes sense to us as the goal is quite simple and its easy to say that a message containing the word urgent would garner a higher score for checking urgency. The -1 is a bias. It shifts the point where the answer changes.

This describes the basic shape of a perceptron; take input numbers, calculate a weighted sum, and apply a rule to produce an output.

Now we did this as a basic example for 2 weights and 2 inputs but for n inputs it’d be more like this:

```
score = weight[0] × input[0]
      + weight[1] × input[1]
      + …
      + weight[n-1] × input[n-1]
      + bias
```

This sum evidently mirrors the dot product computation.

Now imagine doing that same weighted sum calculation thousands of times using the same input vector where each calculation has its own learned weights, which means we’re taking the same input numbers and combining them differently each time.

We can put all these sets of weights into a matrix, where each row holds the weights for one calculation as we've done before. Then multiplying that matrix by our input vector gives us all the weighted sums together. Now each row gives us one number, and those numbers together make up our new output vector.

This is pretty much what we mean by a linear projection, in Llama 3.2 1B model an FFN expansion projection takes the token's 2048 values and produces 8192 values so we now have a larger vector containing different learned combinations of the input features, which gives the network more room to process them.

Remember though, in our urgency example we took the weighted sum and checked whether it was above zero to get a yes or no. Llama's FFN keeps numerical values and uses a form of nonlinear processing called SwiGLU.

So now we know how we can take a token's 2048 values, multiply them by a learned weight matrix and get 8192 values back. In Llama's FFN we actually do this twice, once using a matrix called W_up and again using a separate matrix called W_gate. Remember though both of these are working with the same input vector, which is the token's vector after our second RMSNorm, meaning we aren't doing the up projection and then passing its output into the gate projection. They both start from the same input, they just have different learned weights.

Let's call that normalized input vector x, then for one token we have:

$$
\begin{aligned}
x &= 2048 \text{ values} \\
W_up &= [8192, 2048] \\
W_gate &= [8192, 2048]
\end{aligned}
$$

As we talked about before, every row contains one set of weights that we use to calculate one weighted sum, so having 8192 rows means we get 8192 output values. Each row has 2048 weights because we need one weight for each of the 2048 input values.

The actual computations look like this:

$$
\begin{aligned}
up &= x \times W_up^{T} \\
gate &= x \times W_gate^{T}
\end{aligned}
$$

We transpose the weight matrices here because we're treating x as a row vector, so the dimensions become:

$$[1, 2048] \times [2048, 8192] = [1, 8192]$$

```
up[0] =
    x[0]    × W_up[0, 0]
  + x[1]    × W_up[0, 1]
  + ...
  + x[2047] × W_up[0, 2047]
```

Then for up[1] we use the next row of weights, but still the exact same input vector, and we keep going until we've calculated all 8192 values. The gate projection does the same thing using its own weights. Also remember our urgency perceptron had a bias, but these Llama projections don't, so here we just calculate the weighted sum.

The up projection is called up because we're expanding from 2048 values to 8192, pretty much giving the network more intermediate features to work with. This doesn't mean we're copying the original values or just adding zeros to make the vector bigger, each new value comes from a learned combination of the input values. Meaning we now have a larger space to process these different combinations before bringing the vector back down to its original size. Just like with our hidden states though, we can't assume every position has a neat English meaning that we can point to.

The gate projection also gives us 8192 values, but we use these to control how the values from the up projection contribute to the next computation. One thing to remember here is that the gate's weights are fixed during inference, but its output values aren't, because those values depend on the token vector we put in. So the same learned matrix can produce different gate values for different tokens.

Before we combine the two vectors, we apply an activation function called SiLU to each value in the gate vector. An activation function is pretty much a function we apply to the result of a weighted sum, like in our urgency example where we checked whether the score was greater than zero to give us a boolean. With SiLU we keep a numerical value, and the calculation is:

$$\text{SiLU}(g) = g \times \text{sigmoid}(g)$$
$$\text{sigmoid}(g) = 1 / (1 + e^{(-g)})$$

$$\text{SiLU}(g) = g / (1 + e^{(-g)})$$

Where g is one value from our gate vector. The sigmoid part gives us a number between 0 and 1, then we multiply that by the original gate value.

Let’s go through an example, say one of our gate values is 2:

$$\text{sigmoid}(2) = 1 / (1 + e^{(-2)}) \approx 0.8808$$
$$\text{SiLU}(2) = 2 \times 0.8808 \approx 1.7616$$
$$\text{sigmoid}(-2) = 1 / (1 + e^{2}) \approx 0.1192$$
$$\text{SiLU}(-2) = -2 \times 0.1192 \approx -0.2384$$
$$\text{sigmoid}(0) = 0.5$$

$$\text{SiLU}(0) = 0 \times 0.5 = 0$$

So you can see that a positive value can pass through fairly strongly, whereas a very negative value gets pushed close to zero. It’s also a smooth function, so during training we can calculate gradients through it to help adjust the weights.

Now that we’ve applied SiLU to the gate vector, we multiply the result by our up vector element by element. Meaning we take the first value from each vector and multiply them, then the second value from each, and keep doing that for all 8192 values.

$$\text{intermediate}[j] = \text{SiLU}(\text{gate}[j]) \times \text{up}[j]$$

Remember this isn’t a dot product, we don’t add the results together, so we still have 8192 values afterwards.

Let’s use smaller vectors so we can actually see the whole computation:

$$\text{gate} = [2, -2, 0]$$
$$\text{up} = [3, 4, 5]$$
$$\text{SiLU(gate)} \approx [1.7616, -0.2384, 0]$$

```
intermediate[0] =  1.7616 × 3 ≈  5.2848
intermediate[1] = -0.2384 × 4 ≈ -0.9536
intermediate[2] =  0      × 5 =  0
```

intermediate ≈ [5.2848, -0.9536, 0]

This is where the idea of a gate comes from, we have one branch producing values and another branch producing factors that scale those values based on the input. Looking at the example, the first up value got amplified, the second changed sign and became smaller in magnitude, and the third became zero. So don't think of this as a strict on/off gate, and don't assume the multiplier is always between 0 and 1 either. That's only true for the sigmoid part, the full SiLU output can be negative or greater than 1.

This combination of SiLU and the elementwise multiplication is called SwiGLU. Pretty much, it lets the contribution of a feature from the up branch depend on a feature calculated by the gate branch, with both coming from the same token's input vector. These relationships are learned during training.

The nonlinear computation is important here because if we only multiplied by one learned matrix to expand the vector, then another learned matrix to bring it back down, we could combine those matrices into one and get the same result. So adding SiLU and multiplying the two branches gives us more ways to process the input than a single linear projection could.

Now we've got our processed vector of 8192 values, but remember the token's existing hidden state has 2048 values, and we eventually need to add our update back to it through the residual connection. So we use another learned matrix called W_down:

$$W_down = [2048, 8192]$$

$$down = intermediate \times W_down^{T}$$

The dimensions are:

$$[1, 8192] \times [8192, 2048] = [1, 2048]$$

```
down[0] =
    intermediate[0]    × W_down[0, 0]
  + intermediate[1]    × W_down[0, 1]
  + ...
  + intermediate[8191] × W_down[0, 8191]
```

Then we do this for every row until we have all 2048 output values. Bringing the vector down doesn't mean we just delete the extra values, we're using learned weights to mix the 8192 intermediate values into a smaller vector.

Lastly we add this update back to the token's existing hidden state through the residual connection. One detail here, we add it to the hidden state from before the second RMSNorm, because the normalized vector was what we used inside the FFN branch. So we need to keep that original hidden state available for the addition.

Putting the whole computation together:

$$
\begin{aligned}
x &= \text{RMSNorm}(\text{hidden_state}) \\
\text{up} &= x \times W_{up}^{\top} \\
\text{gate} &= x \times W_{gate}^{\top} \\
\text{intermediate} &= \text{SiLU}(\text{gate}) \odot \text{up} \\
\text{down} &= \text{intermediate} \times W_{down}^{\top}
\end{aligned}
$$

$$\text{new_hidden_state} = \text{hidden_state} + \text{down}$$

Where $\odot$ just means the elementwise multiplication we went through above.

We've been looking at one token to make it easier to follow, but for the whole prompt we can put the token vectors into rows of a matrix and calculate them together. So the expansion projections go from [token_count, 2048] to [token_count, 8192], then the down projection returns us to [token_count, 2048]. Every token position uses the same weights within this layer, but each row is processed independently, meaning we aren't mixing information between token positions during these FFN computations.

So now we've gone through all the computations inside one transformer layer, but remember from our architecture diagram that our Llama 3.2

1B model has 16 of these layers, meaning what we've just finished is layer 0, and we still have layers 1 through 15 to go through.

The way this works is pretty much taking the hidden states we got after the FFN's residual add and passing them into the next layer, where we do the same sequence of operations again. We don't go back to the embedding table and start over, the updated vectors from the previous layer become the input for the next one. So when layer 1 calculates its queries, keys and values, it's calculating them from representations that have already been processed by layer 0.

Remember though, having the same operations doesn't mean every layer uses the same weights. Each layer has its own learned Q, K, V and output projection matrices, its own RMSNorm weights, and its own gate, up and down projection matrices. Meaning the FFN weights are shared between token positions within one layer, like we mentioned before, but they're different from one layer to another. So each layer gets to work with what the earlier layers have already computed, attention can combine information between token positions again, then the FFN processes the resulting features within each position. We aren't manually assigning each layer a specific job, their different weights were learned together during training to make this whole sequence useful for predicting the next token.

One thing to keep in mind is that for our prompt, the hidden-state matrix still has the same shape after every complete layer, `[token_count, 2048]`. The numbers inside it have changed, but each token still has a 2048-value vector, which lets us pass it straight into the next layer. These are still internal representations at this point, we haven't picked an output token after each layer.

Then as per the architecture diagram we add the final RMSNorm before an output.

So now we've gone through all 16 transformer layers and applied the final RMSNorm, but remember what we have at this point is still a vector of 2048 numbers for a token, these numbers contain the information the model has processed but they aren't directly telling us what token should come next. For that we need the LM head, where LM pretty much means language modeling, and the head is the part that

turns our internal representation or vector into scores/probabilities for possible output tokens.

Remember our model has a vocabulary of 128256 tokens, so we need to go from 2048 hidden-state values to 128256 scores, one for every possible token ID. For our Llama 3.2 1B model the LM head shares its weights with the input embedding table, this is called weight tying. So at the start we used a token ID to grab one row, whereas here we compare the final hidden vector against every row of that same table to get the vocabulary scores, they're two operations using the same learned numbers and we dont need another copy of the weights. We use a separate learned weight matrix for LM head:

$$W_lm_head = [128256, 2048]$$

Each row belongs to one vocabulary token and contains 2048 learned weights, meaning we can take the dot product of our final normalized vector with each row to get a score for that token. These weights were learned during training along with the rest of the model, so the earlier layers and the LM head have been trained together to make useful next-token predictions.

For generating the next token, we use the vector from the last position in the prompt. Remember causal masking means that position can gather information from all the positions before it as well as itself, so by the time it reaches the end of the model its vector can contain information from the whole prompt.

Each position's output is used to predict what comes after that position, meaning if we want to continue the prompt, it's the last position's prediction we need. Our engine therefore only runs this final normalization and LM head calculation on the last prompt position.

Let's call that final normalized vector h:

$$h = [1, 2048]$$
$$logits = h \times W_lm_head^{T}$$

The dimensions become:

$$[1, 2048] \times [2048, 128256] = [1, 128256]$$

And looking at just the score for a vocabulary token with ID v:

```
logits[v] = h[0]    × W_lm_head[v, 0]
          + h[1]    × W_lm_head[v, 1]
          + …
          + h[2047] × W_lm_head[v, 2047]
```

So it’s the same weighted sum computation we’ve already seen, just now each result has a direct connection to a vocabulary token. There’s no bias in this projection either.

This also connects back to why we spent so much time updating the hidden states through attention and the FFN, changing those hidden-state values changes the dot products we calculate here, meaning it changes which vocabulary tokens receive higher or lower scores.

The scores we get are called logits. They can be positive or negative and they aren’t probabilities yet, so a score of 5 doesn’t mean a 50% chance or anything like that. What we can say is that a higher logit gives that token a higher ranking for this particular next-token prediction.

Now we need to actually choose a token, and in our engine we use argmax. This pretty much means finding the position of the largest value in the logits vector. One thing to keep in mind is that we want the index of that value, because the index is the token ID, the score itself isn’t what we send to the tokenizer.

Let’s use a pretend vocabulary with only five tokens so we can see this clearly:

Token ID:	0	1	2	3	4
Logit:	0.4	-1.1	3.7	2.0	-0.3

The largest score is 3.7, but our chosen token ID is 2, because that’s the position where we found the largest score.

```
max(logits)    = 3.7

argmax(logits) = 2
```

For our actual model we do the same thing across all 128256 scores. The basic idea can be written as:

```
next_token_id = 0

highest_score = logits[0]


for each remaining token ID:

    if logits[token_id] > highest_score:
        highest_score = logits[token_id]
        next_token_id = token_id
```

This way of choosing the highest-scoring token is called greedy decoding. For the same logits it picks the same highest-scoring token each time, we aren't randomly sampling from the possible outputs.

You may remember softmax from attention, and we can also apply it here if we want to convert the logits into probabilities:

```
probability[v] = exp(logits[v]) / sum(exp(logits))
```

Where the sum includes every vocabulary token. But for argmax we don't actually need to calculate this, because softmax preserves the ordering of the scores, meaning whichever token had the highest logit will also have the highest probability. So our engine can just find the largest logit directly.

Lastly, the token ID we chose needs to be decoded using the tokenizer so we can get its text or bytes. Remember a token could be a whole word, part of a word, punctuation or something else, so choosing one token doesn't necessarily mean we've generated a whole word.

That chosen token becomes the next position in the sequence, and when generation continues, the model processes that new position and uses its final vector to predict the token after it.

Decode:

Prefill vs.Decode:

So up until now we've pretty much been looking at what happens when we give the model an input prompt, turn it into token IDs, get their embedding vectors and pass them through all 16 transformer layers, then use the final RMSNorm, LM head and argmax to choose the next token. This initial phase of processing the prompt is called prefill, and it's what gives us our first generated token.

Remember though, that first generated token usually isn't the whole answer, it might just be a word or even part of a word, so now we need to keep generating. The token we just chose becomes part of the sequence, and the model needs to use it along with the earlier context to predict the token after that. This repeated phase is called decode.

Let's use a small example where each letter stands for one token:

Input prompt: [A, B, C]

During prefill we process A, B and C through the model, and the final vector at position C is what we use to predict the next token. Say we choose D:

Prefill:
[A, B, C] → predict D

Our sequence is now:
[A, B, C, D]

Now D is the latest token, so we need to process its position through the model to predict what comes after it. Say that gives us E, then we process E to predict the next one, and this keeps going:

Decode step 1:
Process D with the earlier context → predict E

Decode step 2:
Process E with the earlier context → predict F

Decode step 3:
Process F with the earlier context → predict the next token

One thing to keep in mind here is that choosing D during prefill doesn’t mean D has already gone through the transformer layers, we only just selected its ID from the vocabulary scores. It still needs its own embedding lookup and forward pass before we can use its final vector to predict E.

The main difference in how we perform these phases is that during prefill we have all the prompt tokens available, meaning within each layer we can calculate their representations together using matrices. Causal masking still applies though, so a position can only attend to itself and earlier positions even when we calculate them together.

During ordinary decode, for one sequence we process one new token position at a time, because we need to choose that token before we can feed it into the next step. It still goes through all 16 layers, including attention and the FFN, we haven’t skipped those computations just because we’re only working on one new position.

So if our prompt had N tokens, the hidden-state matrix during prefill has shape:

[N, 2048]

Whereas the new hidden state we process during each decode step has shape:

[1, 2048]

Remember though, having one row here doesn’t mean the model only has one token of context. Attention still needs information from the earlier positions, and this is where the KV cache comes in, it lets us keep the keys and values we already calculated so we can use them again while processing the new token.

Also, decode here means this phase of generating further tokens, which is separate from tokenizer decoding, where we convert token IDs back into text.

Why cache?:

So now we know that prefill processes our prompt and decode keeps generating from there, but we haven't really explained how we can process just the new token while still using the earlier context. The most straightforward way would be to take the entire sequence we have so far and run it through the model again every time we want another token.

Using our example from before:

```
Run [A, B, C]       → predict D
Run [A, B, C, D]    → predict E
Run [A, B, C, D, E] → predict F
```

This would work, but remember everything that happens inside one forward pass, we need the embedding lookup, the normalization, Q/K/V projections, attention, the FFN and the residual additions, repeated through all 16 layers. Meaning every time we add one token we would also be doing all that work again for the earlier positions.

For a short example like this it might not seem like a big deal, but imagine having a prompt with thousands of tokens and then generating hundreds more, we'd keep calculating the same earlier positions over and over just to predict one new token.

The reason we can avoid this comes back to causal masking. Remember a token position can only attend to itself and positions before it, so adding something after that position doesn't give it any new information it's allowed to use.

Let's look at what each position can attend to in our original prompt:

```
A can attend to: A
B can attend to: A, B
C can attend to: A, B, C
```

Then we append D:

```
A can attend to: A
B can attend to: A, B
C can attend to: A, B, C
```

D can attend to: A, B, C, D

Notice that nothing changed for A, B or C. Their token IDs and positions are the same, and the set of positions they can attend to is the same. D can use information from them, but they can’t use information from D because it comes after them.

This also holds through the whole model. In layer 0, the earlier positions receive the same input vectors and use the same learned weights, and their attention still only sees the same allowed positions, so they produce the same outputs. The RMSNorm and FFN operations process each position independently too, meaning appending D doesn’t change those computations for A, B or C either.

Then layer 1 receives those same earlier-position outputs from layer 0, so the same reasoning applies again, and this keeps holding through all 16 layers. We don’t need to go back and update the earlier positions when a new token arrives.

So when we process D, we still need the earlier positions’ keys and values for attention, but we don’t need to recalculate them. We can keep the ones we already computed during prefill and reuse them, which is pretty much the motivation behind the KV cache.

Prefill [A, B, C]:
Calculate their representations
Keep their K and V values at each layer
Predict D

Decode D:
Calculate D’s new representation
Use the stored K and V from A, B and C
Include D’s own K and V in attention
Keep D’s K and V for later steps
Predict E

Remember we’re keeping these separately for each layer, because the keys and values calculated in layer 0 are different from the ones calculated in layer 1, each layer has its own weights and receives different hidden states.

This saves us from rerunning the earlier positions through the model, but the new token still has to attend over the accumulated keys and values, so the amount of attention work can still grow as the sequence gets longer. What we've removed is the repeated computation of the earlier token positions, by keeping the results that future positions actually need.

We've now explained why we can reuse earlier computations, but we need to be a bit more specific about what we're actually keeping. The KV cache stores the key and value vectors calculated for the token positions we've already processed, separately for every transformer layer. Remember these are the outputs of the K and V projections, they aren't the learned W_k and W_v weight matrices themselves.

The weight matrices stay fixed during inference, whereas the cached vectors depend on the input sequence. Meaning two different prompts will generally produce different keys and values, even though we use the same model weights to calculate them.

Let's start with one sequence and one layer. In our Llama model we have:

32 query heads × 64 values = 2048 Q values per token

8 key heads × 64 values = 512 K values per token

8 value heads × 64 values = 512 V values per token

Remember this difference comes from grouped query attention, every group of four query heads shares one key head and one value head. So we only need to store eight heads worth of keys and values per token, we don't need to duplicate them to match all 32 query heads.

For our prompt [A, B, C], one layer's cache can be thought of as:

K cache:

[k_A]

[k_B]

[k_C]

V cache:

[v_A]

[v_B]

[v_C]

Each row here has 512 values, containing the eight heads placed beside each other. We have a separate set of these rows for every layer, because each layer calculates its own K and V using its own weights and the hidden states it receives.

A simple way to describe each cache's layout is:

[layer, token_position, kv_feature]

K cache: [16, capacity, 512]

V cache: [16, capacity, 512]

Where capacity is how many token positions we've reserved space for, and the valid length tells us how many positions actually contain results we can use. So reserving space for more tokens doesn't mean we should include those unused positions in attention.

One detail to remember from RoPE is that we rotate Q and K, but not V. In our implementation the cached keys have already had RoPE applied using their positions in the sequence, while the cached values are the outputs of the V projection without that rotation. Meaning when we read an earlier key from the cache later on, we don't rotate it again.

We also don't need to keep the earlier queries for future decode steps. Remember the query represents what the position currently being processed is looking for, so when we process a new token we calculate its new query and compare it with the stored keys. The earlier queries were used to calculate the earlier positions' attention outputs, which we've already finished.

The attention scores and softmax results aren't what we keep in the KV cache either, because the new token has a different query and needs its own scores against the available keys.

Now let's go through one full decode step using the example we've been working with:

Prompt already processed: [A, B, C]

Next token chosen: D

Cached positions: 3

D's position: 3

We're using zero-based positions here, so A is at position 0, B at 1, C at 2 and D at 3. Remember choosing D's token ID doesn't mean we've processed D yet, at this point the cache still only contains A, B and C.

First we use D's token ID to get its embedding vector from the embedding table, giving us our starting hidden state with shape [1, 2048]. Then we enter layer 0 and apply its first RMSNorm, just like we did during prefill.

Using that normalized vector, we calculate D's Q, K and V:

$$\text{normalized} = \text{RMSNorm}(\text{hidden_state})$$

$$q_D = \text{normalized} \times W_q^T$$

$$k_D = \text{normalized} \times W_k^T$$

$$v_D = \text{normalized} \times W_v^T$$

The output shapes are:

$$q_D = [1, 2048]$$

$$k_D = [1, 512]$$

$$v_D = [1, 512]$$

So these are the same projections we've already covered, but now we only calculate one token row. We don't recalculate q_A, k_A, v_A and all the others again.

Next we apply RoPE to D's query and key using position 3. Even though our current input buffer only has one token in it, that token's position in the sequence is still 3, we can't treat it as position 0 just because it's the first row in this new buffer.

q_D = RoPE(q_D, position=3)

k_D = RoPE(k_D, position=3)

Then we put D's key and value into the next cache slot for this layer:

K cache:

[k_A]

[k_B]

[k_C]

[k_D]

V cache:

[v_A]

[v_B]

[v_C]

[v_D]

The earlier keys already have their positional rotations, and D's new key now has its own, so they're ready to use together in attention.

For the current query to attend to itself, we need to include D's key and value as well as the earlier ones. Meaning we now have four valid key positions for D's attention calculation.

Let's look at one query head. It has 64 values, and we select the key head it shares according to the GQA grouping:

kv_head = query_head / 4

Using integer division, so query heads 0 through 3 use KV head 0, query heads 4 through 7 use KV head 1, and so on.

For this one query head, the attention score calculation is:

$$\text{scores} = q_D_head \times K_cached_head^T / \text{sqrt}(64)$$

The dimensions are:

$$[1, 64] \times [64, 4] = [1, 4]$$

So we get four scores, one comparing D's query with A's key, one with B's key, one with C's key and one with D's own key.

This is different from prefill, where each head calculated scores for multiple query positions. Here we have one query position, but it still needs to compare against all four available key positions.

In this one-token decode case, every valid cached position is either earlier than D or D itself, so there aren't any future positions to mask. We still need to restrict attention to the valid cache entries though, otherwise we'd accidentally include unused memory from the space we reserved.

Then we apply softmax across those four scores to get the attention weights. Remember our stable version subtracts the maximum score before taking the exponentials:

```
weight[j] =
    exp(score[j] - max_score)
    / sum(exp(score - max_score))
```

Now we use those weights to combine the matching value vectors:

head_output = attention_weights × V_cached_head

With shapes:

[1, 4] × [4, 64] = [1, 64]

Or looking at the weighted sum directly:

```
head_output =

    weight_A × v_A_head

  + weight_B × v_B_head

  + weight_C × v_C_head

  + weight_D × v_D_head
```

We do this for all 32 query heads, each using the key and value head assigned to its group. Then we place the 32 head outputs beside each other, giving us 2048 values, apply the output projection and add the result back through the attention residual connection.

After that it’s the second RMSNorm, the gate and up projections, SiLU, elementwise multiplication, down projection and the FFN residual add, all the computations we’ve already covered. The KV cache hasn’t replaced any of those operations for D, it has saved us from having to redo the earlier token positions.

Now D’s updated hidden state goes into layer 1, where we repeat the process using layer 1’s weights and layer 1’s cache. Meaning we calculate a new Q, K and V for D at this layer, we don’t reuse its layer 0 projections, because layer 1 receives a different representation and has different learned weights.

This continues through all 16 layers, and by the end D has a key and value stored at position 3 in every layer’s cache. Position 3 is the same throughout the model, we aren’t moving to a new sequence position every time we move to a new layer.

Finally we apply the final RMSNorm, calculate the vocabulary logits with the LM head and use argmax to choose the next token. Say we choose E:

Tokens now processed and cached: [A, B, C, D]

Next token chosen: E

Cache length: 4

E’s position when processed next: 4

One detail here that’s easy to mix up, the cache length counts tokens we’ve actually processed, not every token ID we’ve predicted. So after this step we have D’s K and V stored, but we don’t have E’s yet, those will be calculated when E becomes the input to the next decode step.

So now we’ve seen how the KV cache saves us from recalculating the earlier token positions, but keeping those results takes up GPU memory, meaning we’re trading memory for less repeated computation. The model weights are already using memory, and now we also need space for the keys and values produced by the sequence we’re processing.

Remember we store K and V separately for every layer, and in our model each token produces eight key heads and eight value heads, with 64 values per head. We’re storing these values in BF16, which takes two bytes per value.

So the amount of KV data for one processed token across the whole model is:

2 × number_of_layers × number_of_KV_heads × head_dimension × bytes_per_value

The first 2 is because we store both keys and values. Putting in our model’s numbers:

2 × 16 × 8 × 64 × 2 = 32768 bytes

32768 bytes = 32 KiB per token

Meaning if we had 2048 processed token positions, the K and V data would take:

2048 × 32768 = 67108864 bytes

67108864 bytes = 64 MiB

That’s just the KV cache for one sequence, we still need memory for the model weights, temporary computation buffers and the other things our engine uses.

This also shows one of the benefits of grouped query attention. We still calculate 32 query heads, but only store eight heads worth of

keys and values. If we instead had 32 K/V heads with the same head dimension and storage type, this cache would take four times as much memory.

There’s a difference here between the amount of valid data and the amount of memory we’ve allocated though. In our current implementation we reserve space for a capacity when we initialize the cache, then fill that space as we process tokens. So the valid cache length grows during decode, but we aren’t calling cudaMalloc for one extra token every step, that space is already reserved.

```
capacity = number of token positions we have space for
length   = number of token positions we have actually processed
```

If we later want to serve multiple sequences at once, each sequence also needs access to its own cached results. So longer contexts and more simultaneous requests both increase how much KV memory we need, which is part of why memory management becomes such a big deal in inference engines.

Another thing to remember is that caching doesn’t make every decode step the same amount of work. We’ve removed the repeated projections and other computations for earlier positions, but the new query still needs to read and attend over their keys and values.

```
Processing D → attends over A, B, C, D
Processing E → attends over A, B, C, D, E
Processing F → attends over A, B, C, D, E, F
```

So for the full attention we’re using, a longer sequence means more cached data to read and more attention scores to calculate. Reading the cache uses GPU memory bandwidth, and the model’s weight matrices also need to be read for the new token’s computations, meaning memory movement can become a major part of decode time even though we’ve saved a lot of arithmetic.

Now that we’re keeping data between forward passes, we also need to keep track of which data is valid and when we’re allowed to use it. Otherwise we could have all the attention math right and still get incorrect results because we read the wrong cache positions.

For a simple cache like ours, before processing a new token we need:

```
0 <= length < capacity

new_token_position = length
valid_positions_after_append = length + 1
```

So if length is 3, positions 0, 1 and 2 already contain valid results, and the new token goes into position 3. After that token has been processed through all the layers, we update length to 4.

Remember this position is also what we use for RoPE, we can’t reset it to zero at every decode step. We also need to write into the correct layer’s cache, because using layer 0’s keys while calculating layer 1’s attention would mix results from different computations.

When attention reads the cache, it should only read the valid positions, including the token currently being processed. Any space beyond that is just reserved memory, we can’t assume it contains useful values or even zeros.

Then we need to decide when generation stops. In our engine we check for the tokenizer’s end-of-turn or end-of-sequence token, and we also have a maximum number of generated tokens and a context limit so we don’t keep going forever or run out of cache space.

One detail here, if the token we just predicted tells us to stop, we don’t need to pass it through another decode step. The same applies when we’ve reached the number of output tokens we wanted, there’s no reason to process the final prediction if we aren’t going to ask the model for anything after it.

When the request finishes, our simple implementation frees the K and V allocations. We could also keep an allocation around to use for another request, but then we need to reset the valid length and fill it with the new sequence’s results, we can’t treat the old contents as context for an unrelated prompt.

This is because the cached vectors depend on the whole prefix leading up to each position. Even if two prompts contain the same word somewhere, that doesn’t mean its cached K and V are interchangeable, because its position and the earlier context may be different.

Lastly, a useful way to check that we've implemented this correctly is to compare against an independent implementation, because our cached path and our full forward pass could both agree while sharing the same mistake, like if our RoPE math was wrong in both places. So we can use the matching Hugging Face model with the same checkpoint and exact same input token IDs, compare intermediate tensors as we build each operation, then compare the final logits with a tolerance for floating-point differences. We should try different prompt lengths, cached decode and batched requests as well, so one answer that looks reasonable isnt our whole correctness check.

Batching:

Latency and Throughput:

So now we're kinda done no? I mean we've gone through all of prefill including writing the whole attention and feed forward networks, across all layers, generated a token and did it repeatedly through decode with the help of the KV cache, meaning effectively it can generate answers, so whats next?

We now want it to be able to serve several users efficiently, the classic software problem of scaling. If we expose this through an API that a frontend can call, we need to handle multiple requests efficiently as well, so this is where batching comes in.

How we do so, is the fundamental software problem of parallelism/concurrency as we've seen so many times at this point. To understand the basics if you are not familiar, two important terms to understand are **latency** and **throughput**, lets imagine 3 requests, A,B, and C. If each req takes 5 seconds, then processing them sequentially would mean

A gets its answer after **5 seconds**.
B gets its answer after **10 seconds**.
C gets its answer after **15 seconds**.

Latency here is how long a user waits for their whole answer, whereas throughput is how many requests or generated tokens the engine completes per second. For an LLM we care about time to first token as

well, which includes waiting in the queue and processing the prompt, then the time between output tokens once decode gets going. So total response time also depends on how many tokens we generate, and a batch can improve the total tokens per second while making one user’s first token or later tokens arrive more slowly, like in the example later where starting a new prefill pauses B.

Batching lets the GPU process work from several requests together, making better use of its resources.

At this point we’ve implemented single-request decode meaning we process one new token for each forward pass, so after prefill predicts a token we run that token through all the layers and use the earlier tokens cached KV during attention, so we dont run the prompt tokens through all the projections and FFN again. During prefill we can process the prompt rows together within a layer, but the layers still have to run in order because each one needs the previous layer’s results. The problem is that its possible B and C can be waiting while A decodes even when A isnt using the GPU’s resources fully, so batching lets us put useful work from multiple reqs into the same operations.

With our two-request batch, we stack A’s and B’s predicted next vectors to run through the operations together. Now a larger matrix operation computes both results. The GPU can reuse weight values across the rows and spread operation-launch overhead over more useful work. Each request still uses its own KV history.

A quantitative example of how this aids can be shown like this:

One request: a decode step takes **10 ms**, producing **1 token** → **100 tokens/second**.

Two requests: a decode step takes **12 ms**, producing **2 tokens** → about **167 tokens/second**.

As you can see our decode step increased in time by 20% (2ms) meaning our latency, but our throughput increased by 67%. Clearly there is a tradeoff of latency for throughput and vice versa, it is about finding the correct balance for this, and understanding the functional needs of the engine, if latency is not an issue maybe throughput is more

prioritized and batching of more requests is implemented in the engine.

Buffer reuse:

If you've been following along you'd realize we use alot of buffers, which is temporary space on the gpu where we store numbers produced during a computation. Think of it like a scratch pad or a sticky note, or a rough paper before your final answer. Except its hardware worth thousands of dollars LMAO.

In particular as a token moves through the model, we needed somewhere to put things like hidden state, Q/K/V projections, attention scores and intermediate MLP results. Which are all calculated values from the input.

One approach would be to free it after everytime its used, but its not like we're super constricted for VRAM on the GPU, so considering the fact that we need to repeatedly use this scratch paper or temporary gpu space across all 16 transformer layers it doesn't really make sense to do so. It's like destroying your scratch paper, and starting on a new one despite the fact you were gonna do the same calculations on that paper. In formal terms, this adds a ton of overhead/wasted time we dont need to do.

Instead we keep the allocations available:

Allocate space once
↓
Write a result → use it → overwrite it when it's no longer needed
↓
Repeat across layers and forward passes
↓
Free the space when finished

We still calculate fresh results each time. The storage is what we reuse, and we only overwrite values after their final reader has finished.

The KV cache holds information we need for longer. Temporary K/V projection buffers hold newly calculated vectors until we copy them

into the request’s cache. That cached history must remain available because future tokens will attend to it as we’ve seen for decode.

It’s also important to understand that we can’t just reuse buffers willy nilly, there are times when it is safe to do so, and when it isnt. This lies hand in hand with understanding the order of operations in our architecture and how breaking that, or overwriting values early could corrupt everything.

The general rule is that you can overwrite a buffer once every operation that needs its current values has finished reading them. A “consumer” just means an operation that reads those values. If three people need to copy something from a whiteboard, you have to wait until all three finish before erasing it. Or better said, you have to wait for the data to be consumed from the whiteboard before you use it yourself.

An example of us re-using buffers in our code can be where we hold query values, Q should stay available while attention uses it to calculate scores, and once attention has finished using every Q row, those queries have now completed their job.

We can then use that same space for the output projection’s result. That result should survive until the attention residual addition reads it. After that addition finishes, the space becomes available for the MLP down projection. You can see how these 3 are independent of each other, and so by following the correct order we can safely re-use this space in gpu memory. It happens only after the final consumer has read the important data.

Q values
↓ attention finishes reading Q
Output-projection result
↓ residual addition finishes reading it
MLP down-projection result

This similar re-use patterns also occur in other parts of the code like with normalized_gpu.

Ok so we keep saying we want to parallelize using batching, but how exactly are we going about that, we cant just spawn threads out of thin air, and how would we be working on two completely different requests at the same time?

Prefill gives us each request's first predicted token. We can then batch their decode steps because the requests are independent. Their prefills can happen one after another which is what our implementation does.

Prefill request A → predict token D → save D's ID

Prefill request B → predict token R → save R's ID

While B runs through prefill, A's predicted token ID stays stored. Once both are ready, we have the two next tokens to process, so we finish prefill for both before starting to decode.

For one request, the current decode token has a hidden-state vector containing 2048 numbers, so it takes on the shape of [1,2048]. Lets take a look at the q projection, or creating the Q vector, it would look like multiplying this hidden state vector by the W_q matrix which is [2048,2048] producing q vector of [1,2048]. Now very simply all we do is append a new row to the first requests hidden-state vector with the second requests hidden state vector now creating a matrix. Both rows function independently but we can apply the same matrix multiplications on them so we do so, now getting an output of [2,2048].

If this still confuses you on how the rows stay independent during this math lets use an example. Instead of 2048, lets shrink it to 2 values.

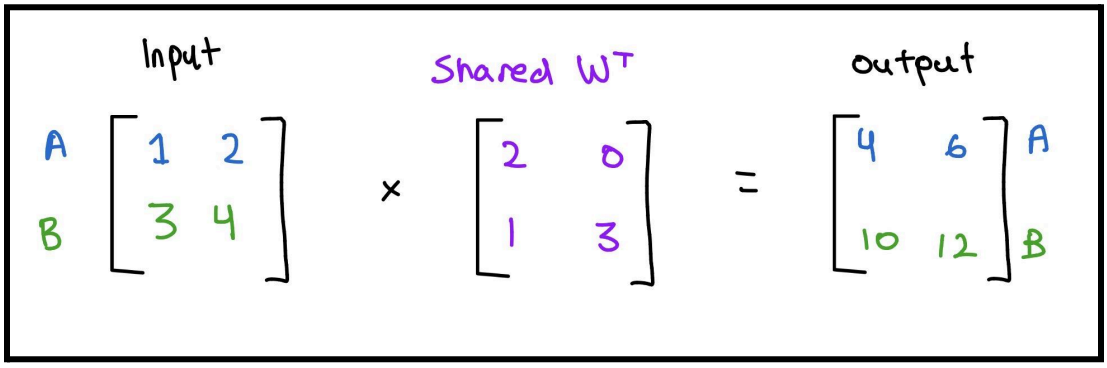
Inputs X:		Weights W^T:
[1, 2]	← A	[2, 0]
[3, 4]	← B	[1, 3]

Computing the matrix multiplication:

A's output: [1×2 + 2×1, 1×0 + 2×3] = [4, 6]

B's output: [3×2 + 4×1, 3×0 + 4×3] = [10, 12]

Each row uses its own input values and the same learned weights. This multiplication never combines A's input with B's input.



Even though we could calculate those two rows through separate matrix multiplications and get the same mathematical results, combining them gives the GPU more work to perform together and it can spread launch overhead across more rows and reuse weight values while calculating those rows, which can improve overall throughput.

Decode step walkthrough:

This next part is definitely a bit more on the verbose side, as it goes through a whole batched decode so you can probably skip, though if you're having a hard time understanding I do recommend you take a look at this.

We've already seen how we can put multiple requests into different rows of a matrix, but I want to actually follow what happens to those rows through one decode step, because there are parts we can calculate together and then there's attention, where we still need to go back to each request's own history.

Say we've run prefill for two requests and we have this:

Request A:
Prompt: [A, B, C]
Predicted next token: D
Cache length: 3

Request B:
Prompt: [J, K, L, M, N]
Predicted next token: O
Cache length: 5

Same idea as our earlier examples, the letters are just standing in for token IDs. We chose D and O, but we still have to actually process them through the model before we can predict what comes after them.

So the first thing we do is collect those two token IDs together, and we also collect the matching cache pointers in that same order, because later when we calculate D's key and value we need to know where they belong.

Token IDs: [D, O]

Matching caches: [A's cache, B's cache]

Position IDs: [3, 5]

The positions come from the cache lengths, remember A already has positions 0, 1 and 2 processed, so D goes at position 3. B has five positions processed, so O goes at position 5, even though it happens to be the second token in this little array we've made.

Then we copy the token IDs and positions to the GPU and do the embedding lookup. This is the same thing we did before, use the token ID to find its row in the embedding table, except now we're gathering two embedding vectors instead of one.

[D's embedding vector]
[O's embedding vector] Shape: [2, 2048]

B having a longer prompt doesn't make its new embedding vector any bigger, it still has 2048 values, its earlier context is sitting in its KV cache. So these two new vectors fit beside each other in the same working matrix just fine.

Now we enter layer 0, do RMSNorm on each row separately, and use the normalized matrix for the Q, K and V projections. We've already covered how those projections work, the difference here is that we're giving the matrix multiplication two input rows.

Q: [2, 2048]
K: [2, 512]
V: [2, 512]

The first row in each of those belongs to D and the second belongs to O. Both rows used the same learned weight matrix for that projection, but each result was calculated from its own input vector, so we still have two separate queries, two separate keys and two separate values.

Next we do RoPE on the new Q and K values, and this is why we bothered collecting the position IDs earlier. D's query and key get rotated using position 3, O's get rotated using position 5. The row number tells us where to find the values in our current buffer, whereas the position tells us where that token is in its own sequence, and those are two different things we need to keep track of.

The old keys in the caches already have their rotations from when we processed them, so we leave them as they are.

Now we've got the new K and V sitting in temporary projection buffers, but future decode steps are going to need them as well, so we copy them into the caches. For D we take row 0, which is 512 values for K and another 512 for V, and put them into A's cache at position 3 for this layer. O's values come from row 1 and go into B's cache at position 5.

These copies happen GPU to GPU, we're moving the calculated vectors from the working buffers into the memory we reserved for each request's history.

In our code this part happens inside a loop over the active rows, so we copy one request's K/V and do its attention, then move on to the other request.

For D, we take its query and calculate attention against A's cached keys, including the new key we just added for D itself. Then it's the same GQA mapping, softmax and weighted combination of values we've already gone through.

D attends to: A, B, C, D

For O, we use its query with B's keys and values:

O attends to: J, K, L, M, N, O

So one attention calculation has four positions available and the other has six. We don't need to make the requests the same length, each attention call gets the valid number of positions for the cache it's reading.

This is also where we keep the two conversations separate, because putting D and O into the same input matrix hasn't given either of them permission to read the other request's history. D's attention only reads A's cache, and O's only reads B's.

After attention, each request has one 2048-value output vector, and we put those into their corresponding rows:

[D's attention output]
[O's attention output]

Shape: [2, 2048]

Then we can use both rows together for the output projection again, since they're both being multiplied by the same output-weight matrix. After that we do the residual addition, adding D's attention update to D's original hidden state and O's update to O's original hidden state.

The second RMSNorm and MLP follow the same idea. We normalize each row, the gate and up projections each give us [2, 8192], then we do SiLU and the elementwise multiplication we covered before, and the down projection brings the result back to [2, 2048] for the next residual addition.

So that's layer 0 done, we have two updated hidden-state vectors and now they go into layer 1, where we repeat this using layer 1's weights and its section of each request's cache. Each layer keeps changing the representations in those two rows, and each layer calculates its own new K and V to store.

D stays at position 3 through all 16 layers, and O stays at position 5. Moving to the next layer doesn't mean we've processed another token, we're still working on those same two token positions.

Once we've finished all the layers, we do the final RMSNorm and pass both rows through the LM head. That gives us:

Logits: [2, 128256]

Row 0: vocabulary scores for what comes after D

Row 1: vocabulary scores for what comes after O

Then argmax looks for the highest score separately in each row. Say D's row picks E and O's row picks P, we return those two token IDs and update the matching requests with their predictions.

We also increase each cache's length once, after the tokens have been processed through all the layers:

```
Request A:
Processed: [A, B, C, D]
Next token: E
Cache length: 4

Request B:
Processed: [J, K, L, M, N, O]
Next token: P
Cache length: 6
```

Remember we've only chosen E and P here, their keys and values aren't in the caches yet. If both requests continue generating, the next decode call takes [E, P], looks up their embeddings, and uses positions [4, 6] as it goes through this process again.

Static Batching:

So now that we know how batching works, both mathematically and having done a walk through of a code. We can now talk about how theres actually different types of batching. We'll be looking at 2 in our case, static and dynamic batching.

So say our inference engine has 2 sequence slots, which means it can keep 2 req active at once, i.e 2 reqs can be batched together. Each occupied slot tracks a request and its KV cache, while additional requests wait in a queue.

So Imagine A and B have already been prefilled:

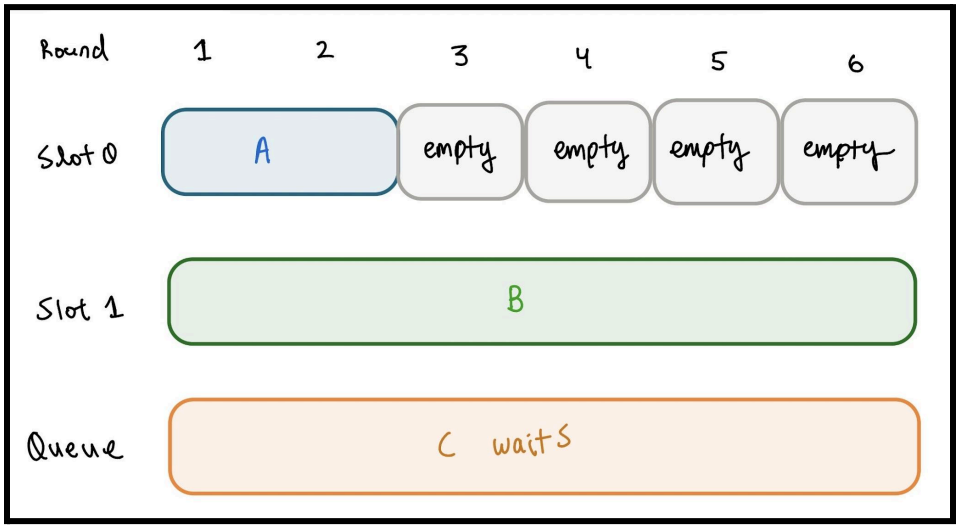
- A needs another **2 decode steps**.
- B needs another **6 decode steps**.
- C is waiting to start.

Each decode step produces one further token prediction for each active request.

With **static batching**, we admit A and B as a group, then wait for that whole group to finish before admitting C. So what that means is here, even though A will finish before B, C will not be admitted into A's slot until B completes despite it being empty. So for 4 time steps, A is done while B keeps going and C is waiting.

Decode round:	1	2	3	4	5	6
Slot 0:	A	A	-	-	-	-
Slot 1:	B	B	B	B	B	B
Waiting:	C	C	C	C	C	C

- means the slot is empty



As you can see theres evidently a problem in that when response lengths vary, for example a yes or no vs writing an essay the first request finishes way before the second one so a third request has to wait pretty unnecessarily.

Continuous Batching:

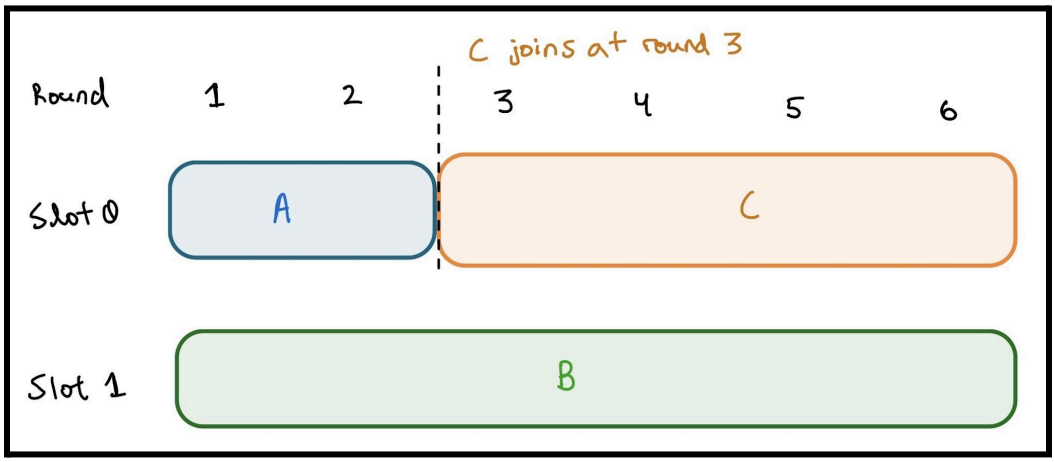
With **continuous batching**, we check for available slots between decode rounds and fill them from the waiting queue.

So going back to our example, Req A still finishes after step 2, but now we admit C into its freed slot before the next decode round, pretty much a better version of what we were doing before with static batching. Diagramatically as so:

```
Decode round:   1   2   3   4   5   6

Slot 0:         A   A   C   C   C   C ...

Slot 1:         B   B   B   B   B   B
                        ↑
              C has joined the batch
```



C continues for however many steps it needs while B keeps its existing slot and KV history, so when C joins, we process C's current token alongside B's current token. This means batch membership changes during generation and where the extra logic behind developing continuous batching comes into play.

We must develop a scheduler, which is the code that manages batch membership during decode. The scheduler, in between decode steps repeatedly does the following actions:

Admit waiting requests into available slots, running prefill for each new request.

Collect the active requests, packing their latest token IDs and matching cache pointers into the decode inputs.

Run one batched decode step, producing a prediction for each active request.

Handle completions, checking stop tokens and limits, returning finished responses and releasing their slots.

Repeat until the waiting queue and all slots are empty.

The main difference you ought to know between static and continuous batching is that Static batching allows admission when the entire group has finished. Continuous batching allows admission whenever a slot becomes free.

Req identity

In our batching implementation theres 3 different things that we need to keep track of:

Request identity: which job this is, such as A or B. Its prompt, generated tokens and token budget belong to that request.

Sequence slot: where that active request's KV cache and slot assignment are kept. The request holds its slot until it finishes.

Packed batch row: where we put its current token in this particular decode call. We rebuild this arrangement each iteration.

For example, initially A and B might line up like this:

```
Decode input row 0 → slot 0 → request A
Decode input row 1 → slot 1 → request B
```

Btw, at this point it just happens to be that the row and slot matches, this is not always the case. Now say A finishes and suppose B is the only request active for the next call:

Slot 0 → empty
Slot 1 → request B

When we build the next decode input, we collect only the active requests, placing them next to each other without empty rows. So B becomes the first and only row:

Decode input row 0 → slot 1 → request B

B's KV cache stays in slot 1. We've simply placed B's current token at row 0 of the temporary input matrix, and active_slots is what remembers this relationship

active_slots = [1]

Meaning: Batch row 0 belongs to sequence slot 1

The decode call returns predictions in that same row order. If it returns: next_tokens = [42], Token 42 belongs to B. To find its owner, we follow:

Output row 0

↓ active_slots[0]

Slot 1

↓ slot_requests[1]

Request B

Later, another request could take the freed slot 0, and the packed rows could change again. The request keeps its identity while its current batch-row position depends on which other requests are active.

Finished req slots:

So now that we can keep multiple requests going together, we also need to know when one of them is actually finished, because otherwise we'd just keep generating tokens and that request would keep holding onto its slot while other requests are waiting.

One way a request can finish is if the model predicts a special stop token. Our tokenizer has an end-of-turn token and an end-of-sequence token, and if we get either of those we finish the request without adding that token to the response we're going to display.

We also have the output-token budget, which is pretty much the maximum number of new tokens we're allowing that request to generate. So if we ask for 20 new tokens, it can stop earlier if it predicts a stop token, but we won't let it keep generating past 20.

Remember though, the first token we predict during prefill is already part of that output. We can't start counting from the first decode call because we would be leaving out a token we've already generated. Say our budget is 3 and we're using the same example as before:

```
Prefill [A, B, C] → predict D
Generated tokens: [D]

Process D → predict E
Generated tokens: [D, E]

Process E → predict F
Generated tokens: [D, E, F]
```

At this point we've reached the budget, so we finish the request and keep D, E and F as its response. There's no reason to send F through the model again, we only needed to do that if we wanted to predict something after F. This also comes back to what we said about the cache length, we've generated F's token ID but we haven't calculated its keys and values.

The other limit is the context size, which in our engine is 2048 positions. This includes the prompt as well as the generated tokens, so if the prompt already takes up 2040 positions we only have room for another 8, even if the user asked for more than that. We account for this when setting the request's token budget, taking whichever is smaller out of the requested amount and the space left after the prompt.

Once any of these stopping conditions is reached, we display the response and release the request's sequence slot:

```
slot_requests[slot] = -1;
caches[slot].length = 0;
```

The -1 tells the scheduler that nobody owns this slot anymore, so it's available for another request when the batching rules allow one to enter. Then we reset the cache length to zero because the next request starts with none of the old request's context.

We're keeping the actual K and V allocations around though, meaning that space is still reserved on the GPU and we can use it again. Resetting the length only changes how much of that space we consider valid, it doesn't go through the allocation and erase every number.

Say the old request had five cached positions, where each box here stands for one position's cached vectors:

```
[old₀, old₁, old₂, old₃, old₄]
length = 5
```

After resetting it, the memory can still look like this:

```
[old₀, old₁, old₂, old₃, old₄]
length = 0
```

Those numbers are physically still there, but from the engine's point of view there is no valid history for the new request yet. Remember attention only gets to read the valid positions we tell it about, we aren't letting it go through the whole allocation just because the space exists.

Now say the next request has a two-token prompt. During prefill we calculate its own keys and values and write them into positions 0 and 1 in each layer's cache:

```
[new₀, new₁, old₂, old₃, old₄]
length = 2
```

So now positions 0 and 1 contain the new request's history, while the old values after that are outside the valid range. They won't be included in its attention calculations.

When we process the next token, its new K and V go into position 2 before attention uses that position:

[new_0, new_1, new_2, old_3, old_4]

The old values at position 2 have been overwritten by the time we allow attention to read it, and we follow that same order for each new position as the request continues. Positions 3 and 4 can keep their old contents until we need them, because they remain outside the history this request is allowed to use.

Memory:

So now that we're keeping a cache for each active request, we should also look at how much GPU memory we're actually reserving, because adding another request means we need somewhere to keep its history as well.

Remember from our earlier calculation, one processed token needs 32 KiB of KV storage across all 16 layers. Our cache capacity is 2048 positions, so for one sequence we reserve:

2048 × 32 KiB = 64 MiB

With both sequence-slot caches initialized, that becomes:

Slot 0: 64 MiB
Slot 1: 64 MiB

Total reserved KV memory: 128 MiB

That's the KV memory specifically, we still have the model weights and the temporary buffers we use during the computations. The weights are shared between the requests, so we load one copy of the model and use those same weight matrices when processing both input rows. Each request needs its own cached results because those depend on its prompt and context.

There are a couple of counts here that we need to keep separate. The capacity of 2048 tells us how many token positions one cache has space for, whereas its length tells us how many positions contain valid results for the request currently using it.

Say A has processed 10 tokens and B has processed 20:

Slot 0:
Capacity: 2048
Valid length: 10

Slot 1:
Capacity: 2048
Valid length: 20

We've only filled 30 positions across the two caches, but we've still reserved the full 128 MiB. We allocated enough space for those sequences to grow, and we keep filling that space as more tokens get processed.

Then there's the active request count, which tells us how many slots currently have a request assigned to them. If A finishes and nobody replaces it yet, we only have one active request, but both cache allocations are still there. As we covered before, finishing A resets its valid length and releases its slot, so the next request can use that same allocation.

This also means having more requests waiting in the queue doesn't mean we've allocated a full KV cache for every one of them. We have space for two active sequences in this setup, and waiting requests get access to those slots as they become available.

Another thing we need to be specific about is what happens when a new request enters, because so far we've mostly been looking at the decode steps where we can put two latest-token vectors into one matrix.

Our prefill still handles one prompt at a time. At the start, we prefill A, then prefill B, and once their first predicted tokens are ready we can collect those tokens and decode them together.

The same thing happens when we refill a slot later. Say A finishes while B is still generating, and C is waiting. We admit C into A's old slot, but C needs its prompt processed before it can join the decode batch, so B waits while that prefill runs.

Decode A and B
A finishes

Prefill C

B waits during this prefill

Decode C and B
Decode C and B
...

B keeps its KV history during that pause, so once C is ready we can continue B from where it left off. If C has a long prompt, though, processing it can create a longer gap between B's output tokens.

So when we call this continuous batching, we're talking about the scheduler checking for freed slots between decode iterations and letting waiting requests enter before the whole previous group has finished. Prefill and decode still run one after another in this implementation, and getting them to overlap is something we can work on separately later.

Paged KV Cache & Attention

Up until this point each sequence had its own kv cache allocation with room for all 2048 possible token positions, which made appending the next token pretty straightforward because we already knew where to put its kv. However we were deciding how much memory belonged to the sequence before knowing how much it would actually need. For our model, one processed token needs 32 KiB of KV storage across all 16 layers, so each sequence's allocation was:

$$2048 \text{ positions} \times 32 \text{ KiB} = 64 \text{ MiB}$$

Say we are dealing with a n-sequence engine so we can have n many requests but this doesn't necessarily mean during processing that they end up using all of the allocated memory and thus that memory is in essence wasted there.

With paging you can think of it more as a pool from which we can assign smaller chunks of memory, these chunks would be pages. Say a request exceeds the one allocated page it can be given another one. The scheduler from KV cache is what orchestrates who gets what.

Furthermore, theres another useful thing to mention. An idea you may have had while learning this as I did was why not just start each contiguous cache small and enlarge it? Well unfortunately that isn’t as easy as it may seem, the allocation of memory we make is fixed and not dynamic as you may see with something like an array in python, it has a fixed size, so “Growing” would really mean allocating a larger space, copying everything over, then freeing that old space… yeah alot uglier than just growing. Paging however, lets us add another chunk while leaving the existing data where it is.

Lets say, using the previous system we were contiguously allocating sequences in the order they were given, but it turns out that the first and third req complete before the second, but the fourth requires more space than what was in first or third, this becomes a problem for the scheduler and is another benefit of paging which takes lessons from computer architecture and the concept of virtual memory that I will briefly go over.

The main idea is that the order a program sees data doesn’t have to match where its pieces actually sit in memory. We keep track of the relationship through a mapping.

Say a program has some data occupying twelve consecutive byte addresses. For this example, imagine a page holds four bytes, so we can split that range into three pages:

Page 0: addresses 0–3
Page 1: addresses 4–7
Page 2: addresses 8–11

These addresses are actually virtual addresses, meaning they are just addr in the space the program works with. They dont actually represent where in RAM those physical bytes actually live. The operating system can then create virtual pages, which hold multiple virtual addresses which actually point to physical frames which are available chunks of physical memory.

That relation, i.e the point from virtual to physical is stored in a page table. When the program accesses memory, the hardware uses the

mappings established by the operating system to translate the virtual address into a physical address. An important add-on is that positioning/offset within the page is same for virtual and physical, so in this example say we are looking at byte 10, given each page holds 4 bytes, 10/4 = 2 meaning page 2 and 10 % 4 = 2 meaning offset 2 bytes within this page, both virtually and within the physical frame.

Now lets connect this to our KV cache. We borrow a similar idea but instead of the OS, our code within the engine handles the mapping itself. We divide a sequence's history into logical pages of 16 token positions, and each page holds the K/V vectors for those positions in one layer. So suppose its block table contains:

Logical page 0 → physical KV page 5
Logical page 1 → physical KV page 1
Logical page 2 → physical KV page 7

The history would still follow Positions 0–15 → positions 16–31 → positions 32–47. Its storage can be scattered around the pool. To find position 33, we calculate: Logical page: 33 / 16 = 2 Offset: 33 % 16 = 1. Then we look up logical page 2, get physical KV page 7, and access the second token position inside that page. Now, those two pages don't have to sit beside each other in GPU memory. Maybe the first piece is stored in pool slot 7and the second in slot 2.

By using a block table within the code it also keeps order telling us which part of the requests history is in which slot so when attention needs it we can just follow.

Pretty much the tldr is that the cache is the saved KV data, and paging is how you divide up the space that stores the data, in a way that efficiently and effectively allows for more concurrent work to be done.

Now if it wasnt already clear, lets look what those pages actually contain and how different requests occupy the same pool. Let's keep with the allocations in the above examples for simplicity. Suppose request A has processed 34 tokens. Looking at just layer 0, its saved K/V needs three pages

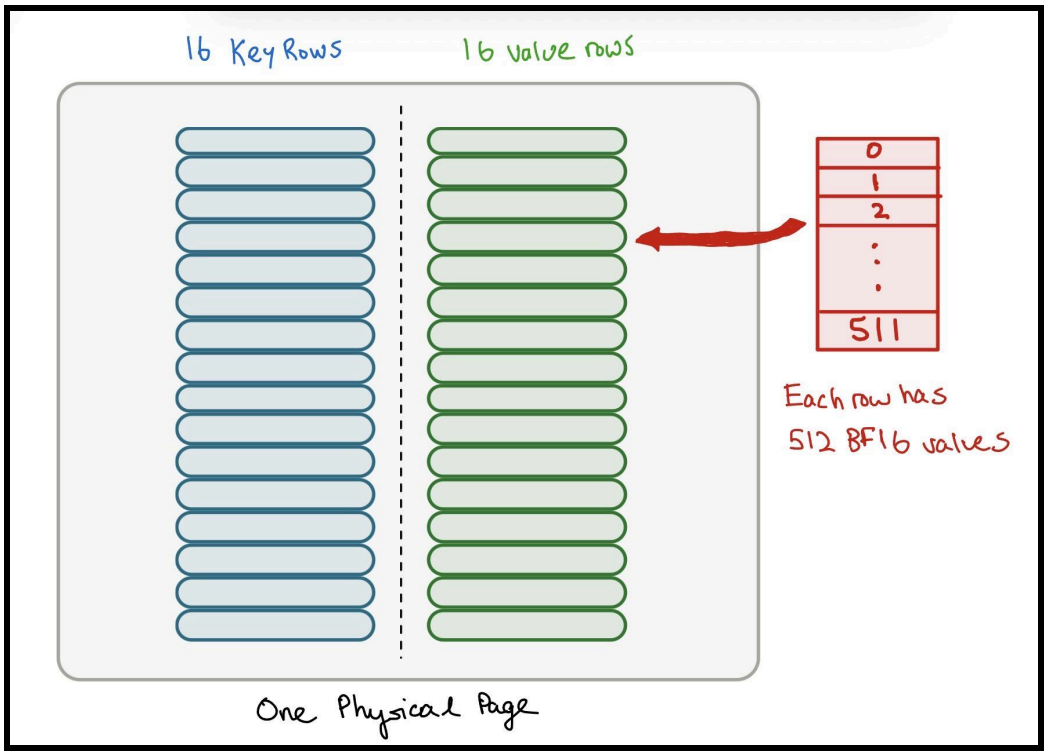
Logical page 0: positions 0–15
Logical page 1: positions 16–31
Logical page 2: positions 32–33, with room for another 14

The block table for req A would then say layer 0: [5, 1, 7]. So essentially we've put the first part of its history in pool slot 5,the next part in slot 1, and the final part in slot 7.

Now suppose request B has processed 20 tokens. It would need two pages for this layer, and we give it different slots, say, layer 0: [2, 6]

So from our shared pool of say from slots 0 - 8, we can see how the requests KV cache data is not contiguous within VRAM, rather one pool of memory for all kv cache data is paged. Their block tables keep the histories separate and ordered

To nail home the concept of growing into the space, lets look at slot 7, it has 2 processed tokens and so if we look back at how a slot is partitioned we have equal space for key and value vector. Thus 2 for key, and 2 for value in our case. Each K or V box represents a whole 512-value vector. The page has space for 16 key vectors and 16 matching value vectors, even though only two positions currently contain valid history.



Those remaining positions stay available for A to grow into. B can receive another free page, but it can't borrow individual unused positions inside A's page.

This is also why we track A's cache length as 34. Three pages provide capacity for 48 positions, but attention should only read the 34 positions we've actually processed.

This example though was only layer 0. Layer 1 produces its own KV, so A has a whole other set of mappings for that layer, and we repeat that across all 16 layers. The shared pool holds the actual vectors and each sequence's block table records where its pieces live.

So now we can look at how attention uses the current token's query to go through its saved history, and we'll keep using request A for this. We've already processed 34 tokens, meaning positions 0–33 are in the cache, so the next token we process goes at position 34. We calculate its Q, K and V, apply RoPE to Q and K, then append its K and V to the cache before running attention, giving us 35 valid positions. The query we're following belongs to position 34, and it can attend to everything from position 0 through 34, including itself.

First the kernel selects A’s block table for layer 0. Remember we upload the block tables in the same packed row order as the queries, meaning each query stays matched with the correct request even when the requests in the batch change. From there we start at logical page 0, which points to physical page 5 in the GPU pool, so that’s where we read the cached keys and values for positions 0–15. For each position we compare the current query with that position’s key to calculate an attention score, which we then use when working out how much its value contributes to the result.

One thing to remember here is that we’re following one query head, and each cached K/V row contains all eight KV heads, so we need to select the matching 64-value slice using the GQA relationship we’ve already looked at. For example, query head 9 uses KV head 2, meaning we take the 64 values belonging to head 2 from the K row, and the matching 64 values from the V row.

Then, once it finishes the first page, it looks up the next logical page. Nothing about the actual query changes here, its just that the next part lives somewhere else in the gpu.

One important thing to note that tripped me up at first as well, when we go from page to page, it doesnt mean that attention restarts. The kernel carries its running attention state across all of its pages, producing one result over the whole valid history. How we’re actually able to do that is Online Softmax which is what i’ll explain next.

Pretty much Online softmax lets us build the attention result one position at a time, keeping a small summary of everything we’ve already read. Through the positions continuing through each page we keep computing values like a Running max which would have the largest score seen so far. We subtract this when calculating exponentials to keep the numbers manageable. Furthermore, the total weight accumulated so far which would be the denominator.As well as the weighted-value sum which are the values accumulated so far, each multiplied by its weight. So the end would look like Attention output = weighted-value sum / denominator.

Let’s go through a small example, say our query gets scores of 1, 2 and 1 when we compare it with three cached keys, and their corresponding values are 10, 20 and 40. We’re using a single number

for each value here so we can follow the calculation, remember though in our actual model each head has 64 values, so we do the same weighted accumulation for each of those components.

Starting with the first position, its score is 1, so our running maximum becomes 1. We subtract that maximum from the score before taking the exponential, giving us:

$$\text{weight} = \exp(1 - 1) = 1$$

This is the weight before normalization, meaning we haven't divided it by the total weight yet. So our denominator starts at 1, and our weighted-value sum starts at:

$$\text{weighted-value sum} = 1 \times 10 = 10$$

Now we move to the second position, which has a score of 2 and a value of 20. Since 2 is larger than our previous maximum, we update the running maximum to 2, but this also changes the scale we're using for our weights. The first weight was calculated by subtracting 1, whereas now we're subtracting 2, so that earlier weight needs to become:

$$\text{previous weight} = \exp(1 - 2) \approx 0.368$$

So how do we fix what we've already accumulated? We multiply both our existing denominator and our existing weighted-value sum by exp(old maximum − new maximum), which here is approximately 0.368. This brings everything we've already processed onto the new scale, and we can do that using just those running totals, without going back and reading the earlier value again.

The second position's own weight is exp(2 − 2) = 1, so after adjusting the old totals and adding this new contribution, we get:

$$\text{denominator} = 1 \times 0.368 + 1 \approx 1.368$$

$$\text{weighted-value sum} = 10 \times 0.368 + 20 \times 1 \approx 23.68$$

From there we move to the third position, which has a score of 1 and a value of 40. Our running maximum stays at 2 because this score is smaller, meaning the earlier totals are already on the correct scale, so we can calculate the new weight and add its contribution directly:

$$\text{weight} = \exp(1 - 2) \approx 0.368$$

$$\text{denominator} = 1.368 + 0.368 \approx 1.736$$

$$\text{weighted-value sum} = 23.68 + 0.368 \times 40 \approx 38.40$$

Now that we've processed all three positions, we divide the weighted-value sum by the denominator:

$$\text{attention output} \approx 38.40 / 1.736 \approx 22.12$$

That division is what normalizes the weights, so the result is a weighted average of our values. We've kept track of the total weight and the weighted contributions as we go, with the running maximum letting us keep those calculations numerically stable.

And this carries straight across our page boundaries too, so when we finish reading physical page 5 and follow the block table to physical page 1, we keep the same running maximum, denominator and weighted-value sum, then continue updating them as we read the next positions. Moving to another page changes where we read the keys and values from, while those totals keep accumulating until we've processed the query's entire valid history.

Conclusion:

Thank you so much for going through this journey with me and reading everything that's spilled out of my brain about the basics of inference. My hope is that you take it from here and strive to learn even more and apply the knowledge and problem solving you learned here to become a better engineer. I hope this wasn't too hard to read ;).